

RAPPORT DE PROJET DE MODULE

Architecture n-tière et Développement Web

LICENCE SCIENCES ET TECHNIQUES

SPÉCIALITÉ : GÉNIE INFORMATIQUE

CONCEPTION ET REALISATION D'UNE PLATEFORME DE COMMUNICATION EN TEMPS REEL DANS DES ROOMS DE CHAT

Réalisé par :

LAMRISSI Bahaa-eddine

Sous la direction de :

Pr. Hicham BENALLA, professeur à la FST de Settat

Soutenu le : 12 Mars 2025

JURY

Pr. Hicham BENALLA, professeur à la FST de Settat

Année Universitaire : 2024-2025

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

وَقُلْ
رَبِّ زِدْنِي عِلْمًا

Dédicace

En préambule, je rends mes louanges à Dieu, source infinie de lumière, de grâce et de bienfaits.

À mon cher père,

Chaque étape de ma vie porte l’empreinte de ta sagesse et de ton amour inconditionnel. Tu m’as transmis les valeurs du travail acharné, de la persévérance et de l’intégrité. Chaque défi surmonté, chaque réussite accomplie, est le reflet de ton héritage précieux et de ton soutien indéfectible.

À ma chère mère,

Tu es l’étoile polaire qui guide mes pas à travers les méandres de la vie. Ton amour, ta force inébranlable et ta bienveillance illuminent mon chemin. Chaque rêve que j’ose poursuivre, chaque victoire que je savoure, est nourri par ton soutien constant et ta lumière inspirante.

À ma précieuse encadrante, Madame Loubna Hamami,

Je tiens à exprimer ma profonde gratitude pour votre accompagnement exemplaire. Votre expertise, votre dévouement et vos encouragements ont été une source inestimable de motivation tout au long de ce projet. Votre guidance bienveillante et votre confiance en mes capacités ont marqué mon parcours académique et professionnel d’une empreinte indélébile.

LAMRISSI

Bahaa-eddine

Remerciements

Je tiens à exprimer ma plus profonde gratitude à toutes les personnes qui ont contribué à la réalisation de ce projet de fin d'études.

Je souhaite également adresser mes sincères remerciements au personnel de la Faculté des Sciences et Techniques pour leur soutien et les ressources mises à ma disposition tout au long de mon cursus. Leur dévouement à l'enseignement et à la recherche a considérablement enrichi mon expérience académique.

Un immense merci à mon encadrant, Monsieur BENALLA Hicham, pour sa disponibilité, ses conseils avisés et son accompagnement tout au long de ce projet. Son expertise et ses encouragements ont été d'une aide inestimable.

Je tiens également à remercier mes camarades de promotion, avec qui j'ai partagé de nombreux moments de travail et de soutien mutuel. Leur amitié et leur collaboration ont rendu cette expérience encore plus enrichissante.

Enfin, je souhaite exprimer ma profonde reconnaissance à ma famille et à mes amis pour leur soutien moral et leurs encouragements constants tout au long de cette aventure académique. Leur présence à mes côtés a été une source précieuse de motivation et de réconfort.

Résumé

Ce projet représente le travail que j'ai réalisé dans le cadre du module d'Architecture systèmes et développement web. L'objectif principal de ce projet était de concevoir et développer une application web de chat en temps réel, offrant une plateforme interactive et sécurisée pour les utilisateurs tout en assurant une gestion efficace des sessions et des communications en ligne.

Dans ce cadre, j'ai commencé par une phase d'analyse préliminaire pour identifier les besoins et les objectifs du projet. Ensuite, j'ai défini les fonctionnalités principales et les contraintes techniques de l'application. Une fois cette étape achevée, j'ai élaboré le design de l'interface utilisateur en utilisant des outils modernes comme Tailwind, avant de passer à la modélisation des bases de données et les dossiers du code pour structurer les composants et les flux de données.

Pour la réalisation, j'ai adopté une approche basée sur les technologies React et Next.js pour le développement frontend, ce qui garantit une interface dynamique et réactive. Pour le backend, j'ai utilisé les API REST de Next.js, associées à une base de données MongoDB, gérée avec Mongoose. Afin d'assurer une expérience utilisateur fluide et un haut niveau de sécurité, j'ai intégré des outils modernes tels que Tailwind CSS pour le design responsive, Socket.io pour les communications en temps réel, et JSON Web Tokens (JWT) pour la gestion des sessions.

Ce projet m'a permis de mettre en pratique les compétences acquises durant mon cursus universitaire et d'explorer des technologies avancées qui répondent aux exigences du développement web moderne, tout en renforçant ma capacité à travailler sur des projets à forte valeur ajoutée.

Abstract

This project represents the work I carried out as part of the N-Tier Architecture and Web Development module. The main objective of this project was to design and develop a real-time web chat application, offering an interactive and secure platform for users while ensuring efficient management of sessions and online communications. In this context, I started with a preliminary analysis phase to identify the project's needs and objectives. Then, I defined the main features and technical constraints of the application. Once this stage was completed, I designed the user interface using modern tools like Tailwind, before moving on to database modeling and organizing the project folders to structure components and data flows.

For the implementation, I adopted a technology-driven approach using React and Next.js for frontend development, ensuring a dynamic and responsive interface. For the backend, I utilized Next.js REST APIs, coupled with a MongoDB database managed with Mongoose. To ensure a smooth user experience and high security, I integrated modern tools such as Tailwind CSS for responsive design, Socket.io for real-time communications, and JSON Web Tokens (JWT) for session management.

This project allowed me to put into practice the skills acquired during my university studies and to explore advanced technologies that meet the demands of modern web development, while also enhancing my ability to work on high-value projects.

ملخص

يمثل هذا المشروع العمل الذي قمت به كجزء من وحدة تطوير الويب. كان الهدف الرئيسي من هذا المشروع هو تصميم وتطوير تطبيق ويب للدردشة في الوقت الفعلي، يوفر منصة تفاعلية وآمنة للمستخدمين مع ضمان إدارة فعالة للجلسات والاتصالات عبر الإنترنت. في هذا السياق، بدأت بمرحلة تحليل أولية لتحديد احتياجات وأهداف المشروع. ثم قمت بتحديد الميزات الرئيسية والقيود التقنية للتطبيق. بمجرد الانتهاء من هذه المرحلة، قمت بتصميم واجهة المستخدم باستخدام أدوات حديثة مثل Tailwind، قبل الانتقال إلى نمذجة قواعد البيانات وتنظيم ملفات المشروع لتكوين المكونات وتدفقات البيانات بشكل هيكلي. في مرحلة التنفيذ، اعتمدت نهجًا يعتمد على التقنيات الحديثة باستخدام React و Next.js لتطوير الواجهة الأمامية، مما يضمن واجهة ديناميكية وسريعة الاستجابة. أما بالنسبة للواجهة الخلفية، فقد استخدمت واجهات REST API الخاصة بـ Next.js مع قاعدة بيانات MongoDB، التي تمت إدارتها باستخدام Mongoose. ولضمان تجربة مستخدم سلسة ومستوى عالٍ من الأمان، قمت بدمج أدوات حديثة مثل Tailwind CSS للتصميم المتجاوب، و Socket.io للاتصالات في الوقت الفعلي، و JSON Web Tokens (JWT) لإدارة الجلسات. أتاح لي هذا المشروع فرصة تطبيق المهارات التي اكتسبتها خلال مسيرتي الدراسية الجامعية واستكشاف تقنيات متقدمة تلبي متطلبات تطوير الويب الحديث، مع تعزيز قدرتي على العمل على مشاريع ذات قيمة مضافة عالية.

Table de matières

Dédicace.....	3'
Remerciements.....	4'
Résumé.....	5'
Introduction générale.....	1
Chapitre 1 : Contexte général du projet.....	3
◦ Introduction.....	4
◦ Présentation de l'organisme d'accueil.....	4
▪ Faculté des Sciences et Techniques Settat.....	4
▪ Missions de la Faculté des Sciences et Techniques Settat.....	5
◦ Cadre général du projet.....	6
▪ Problématique.....	6
◦ Besoins fonctionnels et Besoins non fonctionnels.....	6
▪ Besoins fonctionnels.....	6
▪ Besoins non fonctionnels.....	8
◦ Modèles de données.....	9
▪ Modèle du code de vérification.....	9
▪ Modèle de l'utilisateur.....	10
▪ Modèle de la room.....	12
Chapitre 2 : Étude technique du projet.....	14
◦ Introduction.....	15
◦ Architecture logicielle du système.....	15
◦ Outils utilisés.....	16
▪ Environnement logiciel.....	16
▪ Langages et technologies utilisés.....	17
Chapitre 3 : Réalisation de l'application.....	21
◦ Introduction.....	22
◦ Logo du Site Web.....	22
◦ Présentation des interfaces de l'application.....	22
▪ Interface d'accueil.....	22
▪ Page de connexion.....	23
▪ Page de registration.....	24

Table de matières

▪ Page de récupération du mot de passe oublié.....	28
▪ Page de connexion avec Google.....	32
▪ Page de profil.....	33
▪ Page de création d'une room.....	36
▪ Page de recherche d'une room.....	36
▪ Page de chat dans une room.....	38
◦ Conclusion.....	41
Conclusion générale et perspectives.....	42
Bibliographie.....	43

Liste des figures

Figure 1 : Logo FST Settat	4
Figure 2 : FST Settat	5
Figure 3 : Schéma d'architecture MVC	15
Figure 4 : Accueil	23
Figure 5 : Connexion «message d'erreur»	24
Figure 6 : Registration «choix d'un username»	24
Figure 7 : Registration «champ de l'email»	25
Figure 8 : Registration «champ du mot de passe»	25
Figure 9 : Registration «champ de l'âge»	26
Figure 10 : Registration «champ du sexe»	26
Figure 11 : Registration «champ de nationalité»	27
Figure 12 : Registration «succès»	27
Figure 13 : Récupération du compte «saisir l'email»	28
Figure 14 : Récupération du compte «code reçu sur Gmail»	29
Figure 15 : Récupération du compte «champ du code de vérification».....	30
Figure 16 : Récupération du compte «saisi de code de vérification»	30
Figure 17 : Récupération du compte «saisi de code de vérification»	31
Figure 18 : Récupération du compte «saisi de code de vérification»	32
Figure 19 : Profil «informations personnelles»	33
Figure 20 : Profil «modification des informations»	34
Figure 21 : Profil «modification des informations»	34
Figure 22 : Profil «modification des informations»	35
Figure 23 : Profil «suppression du compte»	35
Figure 24 : Création d'une room	36
Figures 26, 27, 28 : Recherche d'une room «restrictions»	37
Figure 29 : Recherche d'une room	37
Figure 30 : Test des messages «compte1»	38
Figure 31 : Test des messages «compte2»	38
Figure 32 : Test des messages «messages multimédias»	40
Figure 33 : Informations d'une room	40
Figure 34 : Profil d'un utilisateur	41

Abréviations

- **API** : Application Programming Interface
- **BDD** : Base de Données
- **CSS** : Cascading Style Sheets
- **CRUD** : Create, Read, Update, Delete
- **DBMS** : Database Management System
- **GSAP** : GreenSock Animation Platform
- **HTTP** : HyperText Transfer Protocol
- **ID** : Identifier
- **IP** : Internet Protocol
- **JS** : JavaScript
- **JSON** : JavaScript Object Notation
- **JWT** : JSON Web Token
- **MVC** : Model Views Controllers
- **NPM** : Node Package Manager
- **REST** : Representational State Transfer
- **SMTP** : Simple Mail Transfer Protocol
- **URI** : Uniform Resource Identifier
- **URL** : Uniform Resource Locator
- **UI** : User Interface
- **UX** : User Experience

Introduction générale

Dans un monde de plus en plus connecté, les technologies de communication en temps réel sont devenues un pilier fondamental de notre quotidien. Ces outils jouent un rôle essentiel dans le rapprochement des individus, la facilitation des échanges et l'amélioration de la collaboration, que ce soit dans des contextes professionnels, éducatifs ou personnels. La montée en puissance de ces solutions repose sur l'évolution rapide des technologies numériques, mais également sur une compréhension approfondie des besoins des utilisateurs modernes, toujours en quête d'interfaces intuitives, performantes et sécurisées.

Dans le cadre de ma formation en licence à la Faculté des Sciences et Techniques de Settat (FSTS), j'ai eu l'opportunité de réaliser un projet de fin d'études qui s'inscrit pleinement dans cette dynamique technologique. Ce projet vise à concevoir et développer une application web de chat en temps réel. L'objectif principal est de proposer une plateforme moderne qui permet non seulement de connecter les utilisateurs instantanément, mais aussi de répondre à des exigences spécifiques comme la gestion des rooms, la sécurité des échanges et l'optimisation des performances. Ce travail s'inscrit également dans une démarche pédagogique visant à consolider mes compétences techniques et scientifiques acquises durant ma formation, tout en me préparant à affronter les défis réels du milieu professionnel.

L'application que j'ai développée repose sur une architecture robuste et des technologies modernes. Parmi celles-ci, React et Next.js ont été utilisés pour le développement de l'interface utilisateur, offrant une expérience fluide et réactive. MongoDB a été choisi pour gérer les données de manière flexible et évolutive, tandis que Socket.io permet de gérer les communications en temps réel, garantissant ainsi une interaction fluide entre les utilisateurs. En outre, des fonctionnalités avancées comme l'authentification sécurisée (à travers JWT et Next-Auth), la gestion des sessions, la création et la recherche de rooms avec des restrictions personnalisées, ainsi que la gestion des profils utilisateurs, ont été intégrées pour répondre aux besoins variés des utilisateurs.

Ce projet a nécessité une méthodologie structurée. J'ai commencé par une phase d'analyse approfondie des besoins afin d'identifier les attentes spécifiques des

utilisateurs cibles. Cette analyse a permis de définir les fonctionnalités clés de l'application, notamment la gestion des utilisateurs, l'authentification multi-facteurs, les communications en temps réel et l'administration des rooms. La phase de conception a ensuite porté sur l'élaboration de l'architecture technique et fonctionnelle, en mettant l'accent sur l'évolutivité, la sécurité et la simplicité d'utilisation. Enfin, la phase de développement a consisté à implémenter ces fonctionnalités en suivant les meilleures pratiques du développement logiciel.

La réalisation de cette application représente une opportunité précieuse pour moi. Elle m'a permis d'approfondir ma maîtrise des technologies modernes, de comprendre les enjeux liés à la sécurité des données et à la gestion des performances dans un environnement web, et de développer des compétences pratiques en gestion de projet. De plus, elle m'a appris à adopter une approche autonome et méthodique pour résoudre les problèmes rencontrés, tout en gardant un œil sur les attentes des utilisateurs finaux. Au-delà de l'aspect technique, ce projet m'a également sensibilisé à l'importance d'un design centré sur l'utilisateur. Offrir une expérience fluide, intuitive et agréable est aujourd'hui un enjeu majeur dans le développement de toute application. Ainsi, j'ai veillé à intégrer des animations modernes et des interfaces claires tout en maintenant une cohérence dans le design.

Pour présenter de manière exhaustive ce travail, ce rapport est structuré en plusieurs parties. La première partie est dédiée à l'analyse des besoins et à la conception fonctionnelle de l'application. La deuxième partie explore les choix technologiques et explique les raisons qui ont motivé ces choix, en tenant compte des contraintes et des objectifs du projet. Enfin, la dernière partie met en avant les principales fonctionnalités de l'application et les résultats obtenus, accompagnés de démonstrations et d'exemples concrets.

En conclusion, ce projet est bien plus qu'un simple exercice académique. Il reflète une démarche proactive et innovante pour répondre aux besoins d'un monde en constante évolution. À travers cette réalisation, j'espère avoir posé les bases d'une plateforme de communication en temps réel qui pourrait, à terme, être adaptée et étendue pour répondre à des cas d'utilisation encore plus diversifiés. Ce travail constitue également une étape significative dans mon parcours académique et professionnel, me préparant à relever avec confiance les défis de demain.

Chapitre 1 : Contexte général du projet

I. Introduction

L'analyse du projet est une approche stratégique essentielle pour garantir le succès global et la gestion fluide de toutes les phases d'un projet. Une étude approfondie et efficace est généralement synonyme de réussite. Ainsi, notre premier chapitre sera consacré à cette étude, englobant la présentation du projet ainsi que la présentation générale de l'organisme d'accueil, la FSTS, et notre méthodologie de développement.

Les paragraphes suivants détailleront la démarche suivie pour la réalisation de ce projet, en mettant en lumière les objectifs visés et les différentes étapes clés à franchir.

II. Présentation de l'organisme d'accueil

1. Faculté des Sciences et Techniques Settât

La Faculté des Sciences et Techniques de Settât (FSTS) est un établissement universitaire à caractère scientifique et technique, qui fait partie de l'Université Hassan 1er, elle a été créée en mars 1994.



Figure 1 : Logo fst settat

Elle est destinée à s'intégrer dans le pôle technologique et industriel de la région de Chaouia-Ourdigha pour être une pépinière de techniciens et de cadres de haut niveau capables de servir de courroie de transmission entre le technicien supérieur et l'ingénieur concepteur.



Figure 2 : FST SETTAT

2. Missions de la Faculté des Sciences et Techniques Settat

L'objectif de la FSTS est d'offrir aux étudiants une formation adéquate en mettant à leurs dispositions un enseignement théorique et pratique et ce dans le but de réussir leur intégration dans le domaine de travail.

A cet effet la FSTS a pour missions :

- La formation universitaire dans les domaines scientifiques et techniques. La
- formation continue des cadres techniques des industries environnantes. La recherche
- appliquée et les prestations des services pour le développement de l'économie régionale et nationale. Afin de réussir ses missions de formation, la FSTS offre à ses étudiants un enseignement semestriel et modulaire, alliant les aspects scientifiques
- et techniques fondamentaux avec les nouvelles orientations technologiques et professionnelles. Elle dispose d'un corps enseignant qualifié et d'un parc de matériel scientifique d'enseignement et de recherche couvrant plusieurs domaines tels que le Génie mécanique, le Génie électrique, l'Informatique, la Chimie, la Physique, la Biologie, la Géologie, et la Communication.

La FSTS est dotée de six départements d'enseignement et une cellule des langues et de Communication :

- Département de Mathématiques et Informatiques
- Département de Chimie appliquée et Environnement
- Département de Biologie appliquée et Agroalimentaire
- Département de Géologie appliquée
- Département de Génie électrique et Génie mécanique
- Département de Physique appliquée

L'année universitaire est composée de 2 semestres comprenant chacun 16 semaines d'enseignement et d'évaluation.

Avec ses six départements et sa cellule de communication, ses quatorze laboratoires et ses

trente-cinq équipes de recherche, la FSTS se transforme en une véritable pépinière de chercheurs, de techniciens et de cadres de haut niveau, alliant à l'enseignement universitaire de

base une formation professionnalisant. Cette orientation stratégique de la FSTS prend toute son

ampleur avec ses relations qu'elle ne cesse de concrétiser et de développer avec ses

partenaires à l'échelle nationale et internationale. L'ouverture sur d'autres horizons a permis à la FSTS de mieux adapter la matière pédagogique qu'elle propose à l'évolution des sciences et techniques en prenant en considération les exigences du monde professionnel. En parallèle, la FSTS connaît une dynamique remarquable relative aux activités culturelles et sportives ayant un impact positif sur la vie des étudiants tant au niveau personnel que relationnel. La mise en place du bureau d'étudiant, des Clubs et des Associations estudiantines, rivalisant d'ardeur et de compétences, est la preuve tangible que l'enseignement universitaire n'est pas seulement une préparation à la vie mais la vie elle-même.

III. Cadre général du projet

1. Problématique

Comment concevoir et développer une application web de chat en temps réel qui répond aux besoins des utilisateurs modernes en termes de fluidité, de sécurité et d'évolutivité, tout en intégrant des fonctionnalités avancées telles que l'authentification sécurisée, la gestion des sessions, et une communication instantanée performante, dans un contexte de transformation numérique rapide et d'exigences croissantes en matière d'expérience utilisateur ?

IV. Besoins fonctionnels et Besoins non fonctionnels

Les besoins fonctionnels définissent les fonctionnalités spécifiques qu'un système doit offrir pour satisfaire les besoins des utilisateurs. Ils servent de guide pour le développement et la conception du système, en décrivant les actions que le système doit être capable d'accomplir, telles que la gestion des données, les interactions avec les utilisateurs, etc. En revanche, les besoins non fonctionnels définissent les exigences de qualité et de performance que le système doit respecter, comme la sécurité, la fiabilité, la convivialité, les performances, etc. Définir ces besoins dès le départ est essentiel pour garantir que le système développé répond aux attentes des utilisateurs et fonctionne de manière fiable et efficace.

1. Besoins fonctionnels :

1. Authentification et gestion des utilisateurs :

- Permettre aux utilisateurs de s'inscrire avec un nom d'utilisateur, une adresse e-mail et un mot de passe.
- Authentification via des comptes Google pour simplifier l'accès.

- Fonctionnalité de récupération de mot de passe par e-mail.
- Gestion sécurisée des sessions avec cookies et tokens JWT.
- Possibilité pour les utilisateurs de mettre à jour leur profil (nom, photo, âge, sexe, etc.).
- Option de suppression définitive du compte utilisateurs

2. Gestion des rooms :

- Création d'une room par un utilisateur, avec la possibilité de définir :
 - Un nom unique pour la room.
 - Une image ou une animation représentant la room.
 - Une description, un type (jeux, discussions, études, etc.) et des restrictions (âge, sexe).
 - Le nombre maximum de membres pouvant rejoindre la room.
- Recherche d'une room via une adresse IP et un port spécifiques.
- Rejoindre une room uniquement si les restrictions définies sont respectées et que le nombre maximum de membres n'est pas atteint.
- Empêcher un utilisateur de rejoindre une room déjà intégrée.
- Consultation des informations d'une room (créateur, membres, restrictions, etc.).

3. Communication en temps réel :

- Envoi de messages texte instantanés dans une room.
- Possibilité d'envoyer d'autres types de contenus (images, vidéos, fichiers audio, etc.).
- Affichage des messages des autres utilisateurs en temps réel grâce à Socket.io.

4. Gestion des utilisateurs dans une room :

- Afficher la liste des membres connectés dans une room avec leur profil.
- Permettre de cliquer sur un utilisateur pour consulter son profil.
- Autoriser un utilisateur à quitter une room.
- Supprimer une room par son créateur si elle n'est plus nécessaire.

5. Navigation et interface utilisateur :

- Une interface utilisateur intuitive et moderne.
- Une barre latérale (sidebar) dynamique, affichant les principales fonctionnalités et personnalisée selon les besoins de l'utilisateur.
- Interface responsive pour une compatibilité optimale avec les appareils mobiles et desktop.
- Une animation fluide et des transitions interactives grâce à GSAP et framer-motion.

Ces besoins fonctionnels assurent une application performante, intuitive et sécurisée, adaptée aux attentes des utilisateurs et aux standards des applications modernes. Si vous souhaitez des ajouts ou précisions, faites-le-moi savoir !

2. Besoins non fonctionnels :

1. Performance :

- Garantir une latence minimale dans les échanges en temps réel entre les utilisateurs (grâce à Socket.io).
- Assurer une rapidité de réponse pour toutes les requêtes utilisateur, avec un temps de chargement des pages inférieur à 2 secondes.
- Optimiser la gestion des ressources pour les environnements à faible bande passante.

2. Sécurité :

- Chiffrer les mots de passe des utilisateurs avant de les stocker (via bcrypt.js).
- Valider les sessions utilisateur avec des tokens JWT pour garantir leur authenticité.

3. Évolutivité :

- Permettre l'ajout futur de nouvelles fonctionnalités (par exemple, support pour des appels vocaux ou vidéo).
- Adapter l'application pour supporter une augmentation du nombre d'utilisateurs sans compromettre les performances.
- Utiliser une architecture backend modulaire pour faciliter les mises à jour.

4. Fiabilité :

- Assurer une disponibilité de 99,9 % avec des mécanismes de récupération en cas de panne.
- Implémenter des sauvegardes automatiques pour protéger les données utilisateur et les historiques de messages.
- Garantir l'intégrité des données échangées via des contrôles rigoureux.

5. Accessibilité :

- Adapter l'interface pour être utilisable par des personnes avec des besoins spécifiques (grande taille de texte, etc.).

6. Compatibilité :

- Assurer la compatibilité avec les navigateurs les plus courants (Chrome, Firefox, Safari, Edge).

- Optimiser la responsivité de l'application pour une utilisation sur des appareils mobiles, tablettes et ordinateurs.
- Garantir le bon fonctionnement sur différents systèmes d'exploitation (Windows, macOS, Android, iOS).

7. Convivialité :

- Fournir une interface intuitive et simple d'utilisation pour tous les types d'utilisateurs, novices ou expérimentés.
- Intégrer des animations fluides pour rendre l'expérience plus interactive et agréable.
- Inclure des tutoriels ou des guides pour accompagner les nouveaux utilisateurs dans leur prise en main de l'application.

8. Internationalisation :

- Prévoir une structure permettant l'ajout futur de traductions pour différents langages.
- Supporter les formats régionaux (dates, heures) en fonction de l'utilisateur.

9. Gestion des logs et des erreurs :

- Implémenter un système de journalisation (logging) pour surveiller les événements critiques et les activités utilisateur.
- Fournir des messages d'erreur clairs et utiles pour guider les utilisateurs lors de problèmes.

Ces besoins non fonctionnels garantissent que l'application soit non seulement performante et sécurisée, mais également évolutive, conviviale et fiable sur le long terme.

V. Modèles des données non relationnelles :

1. Modèle du code de vérification (VerifCode) :

Ce schéma peut être utilisé pour créer, lire, mettre à jour et supprimer des codes de vérification dans une base de données MongoDB. Il permet de structurer les données de manière cohérente et de faciliter les opérations CRUD (Create, Read, Update, Delete) dans l'application. Par exemple, lorsqu'un utilisateur demande une réinitialisation de mot de passe, un code de vérification est généré et stocké dans la base de données avec une date d'expiration. L'utilisateur doit entrer ce code avant la date d'expiration pour compléter le processus de réinitialisation.

Champs du Schéma

1. email :

- Type : String
- Obligatoire : Oui
- Description : Adresse email de l'utilisateur à qui le code de vérification est envoyé. Ce champ est utilisé pour identifier l'utilisateur et associer le code de vérification à l'email correspondant.

2. code :

- Type : String
- Obligatoire : Oui
- Description : Code de vérification généré pour l'utilisateur. Ce champ contient le code que l'utilisateur doit entrer pour vérifier son email ou réinitialiser son mot de passe.

3. expiresAt :

- Type : Date
- Obligatoire : Oui
- Description : Date et heure d'expiration du code de vérification. Ce champ est utilisé pour définir la durée de validité du code de vérification. Une fois cette date et heure atteintes, le code de vérification ne sera plus valide.

2. Modèle de l'utilisateur (User) :

Ce schéma peut être utilisé pour créer, lire, mettre à jour et supprimer des codes de vérification dans une base de données MongoDB. Il permet de structurer les données de manière cohérente et de faciliter les opérations CRUD (Create, Read, Update, Delete) dans l'application. Par exemple, lorsqu'un utilisateur demande une réinitialisation de mot de passe, un code de vérification est généré et stocké dans la base de données avec une date d'expiration. L'utilisateur doit entrer ce code avant la date d'expiration pour compléter le processus de réinitialisation.

Champs du Schéma

1. username :

- Type : String
- Obligatoire : Oui
- Description : Nom d'utilisateur unique de l'utilisateur. Ce champ est utilisé pour identifier l'utilisateur de manière unique dans l'application.

2. email :

- Type : String
- Obligatoire : Oui
- Unique : Oui
- Description : Adresse email de l'utilisateur. Ce champ est utilisé pour la communication avec l'utilisateur et doit être unique pour chaque utilisateur.

3. password :

- Type : String
- Obligatoire : Oui
- Description : Mot de passe de l'utilisateur. Ce champ est utilisé pour l'authentification de l'utilisateur et doit être stocké de manière sécurisée (par exemple, en utilisant le hachage).

4. bio :

- Type : String
- Obligatoire : Non
- Description : Biographie ou description de l'utilisateur. Ce champ permet à l'utilisateur de fournir des informations supplémentaires sur lui-même.

5. age :

- Type : Number
- Obligatoire : Non
- Description : Âge de l'utilisateur. Ce champ permet de stocker l'âge de l'utilisateur, ce qui peut être utilisé pour appliquer des restrictions d'âge dans certaines rooms.

6. sex :

- Type : String
- Obligatoire : Non
- Enum : ["Boy", "Girl"]
- Description : Sexe de l'utilisateur. Ce champ peut prendre l'une des valeurs suivantes : "Boy" ou "Girl", ce qui permet de catégoriser les utilisateurs en fonction de leur sexe.

7. country :

- Type : String
- Obligatoire : Non
- Description : Pays de l'utilisateur. Ce champ permet de stocker le pays de résidence de l'utilisateur, ce qui peut être utilisé pour des fonctionnalités géographiques.

3. Modèle de la Room :

Ce schéma peut être utilisé pour créer, lire, mettre à jour et supprimer des rooms de discussion dans une base de données MongoDB. Il permet de structurer les données de manière cohérente et de faciliter les opérations CRUD (Create, Read, Update, Delete) dans l'application.

1. `name` :

- Type : String
- Obligatoire : Oui
- Description : Nom de la room. Ce champ est utilisé pour identifier et afficher le nom de la room dans l'interface utilisateur.

2. `ip` :

- Type : String
- Obligatoire : Oui
- Description : Adresse IP de la room. Ce champ est utilisé pour identifier la room de manière unique sur le réseau.

3. `port` :

- Type : Number
- Obligatoire : Oui
- Description : Numéro de port de la room. Ce champ est utilisé pour identifier la room de manière unique sur le réseau en combinaison avec l'adresse IP.

4. `owner` :

- Type : String
- Obligatoire : Oui
- Description : Nom d'utilisateur du propriétaire de la room. Ce champ est utilisé pour afficher le propriétaire de la room dans l'interface utilisateur.

5. `ownerID` :

- Type : `mongoose.Schema.Types.ObjectId`
- Référence : User
- Obligatoire : Oui
- Description : Identifiant unique du propriétaire de la room. Ce champ est une référence au modèle User, ce qui permet de lier chaque room à un utilisateur spécifique.

6. **maxMembers** :

- 1.Type : Number
- 2.Obligatoire : Oui
- 3.Description : Nombre maximum de membres autorisés dans la room. Ce champ est utilisé pour limiter le nombre de participants dans la room.

7. **type** :

- 1.Type : String
- 2.Enum : ["Studing", "Gaming", "Chatting", "Designing", "Other"]
- 3.Description : Type de la room. Ce champ peut prendre l'une des valeurs suivantes : "Studing", "Gaming", "Chatting", "Designing", ou "Other", ce qui permet de catégoriser les rooms en fonction de leur utilisation.

8. **description** :

- 1.Type : String
- 2.Description : Description de la room. Ce champ est utilisé pour fournir des informations supplémentaires sur la room, telles que son objectif ou ses règles.

9. **boysOnly** :

- 1.Type : Boolean
- 2.Valeur par défaut : false
- 3.Description : Indique si la room est réservée aux garçons. Ce champ est utilisé pour appliquer des restrictions de genre à la room.

10. **girlsOnly** :

- 1.Type : Boolean
- 2.Valeur par défaut : false
- 3.Description : Indique si la room est réservée aux filles. Ce champ est utilisé pour appliquer des restrictions de genre à la room.

11. **plus18Only** :

- 1.Type : Boolean
- 2.Valeur par défaut : false
- 3.Description : Indique si la room est réservée aux utilisateurs de plus de 18 ans. Ce champ est utilisé pour appliquer des restrictions d'âge à la room.

12. `joinedUsers` :

- Type : Array
- Description : Liste des utilisateurs qui ont rejoint la room. Chaque élément du tableau contient les champs suivants :
 - `userID` : Identifiant unique de l'utilisateur qui a rejoint la room. Ce champ est une référence au modèle User.
 - `joinedAt` : Date et heure à laquelle l'utilisateur a rejoint la room. Ce champ est automatiquement défini à la date et l'heure actuelles lors de la création de l'entrée.

13. `members` :

- Type : Number
- Valeur par défaut : 0
- Description : Nombre actuel de membres dans la room. Ce champ est utilisé pour suivre le nombre de participants dans la room.

14. `img` :

- Type : String
- Obligatoire : Non
- Description : URL de l'image de la room. Ce champ est utilisé pour afficher une image représentative de la room dans l'interface utilisateur.

15. `messages` :

- Type : Array
- Description : Liste des messages envoyés dans la room. Chaque élément du tableau contient les champs suivants :
 - `userID` : Identifiant unique de l'utilisateur qui a envoyé le message.
 - `username` : Nom d'utilisateur de la personne qui a envoyé le message.
 - `type` : Type de contenu du message. Ce champ peut prendre l'une des valeurs suivantes : "text", "image", "video", ou "audio".
 - `content` : Contenu du message. Ce champ contient le texte ou l'URL du contenu multimédia (image, vidéo, audio) du message.
 - `date` : Date et heure d'envoi du message. Ce champ est automatiquement défini à la date et l'heure actuelles lors de la création du message.

Chapitre 2 : Etude technique du projet

I. Introduction

Dans ce deuxième chapitre, nous visons à présenter les outils, technologies et l'architecture technique de l'application utilisés pour mettre en œuvre notre solution.

II. Architecture logicielle du système

L'architecture Modèle-Vue-Contrôleur (MVC) est une méthode d'organisation de l'interface graphique d'un programme. Elle divise cette interface en trois couches distinctes : le modèle, la vue et le contrôleur, chacune ayant un rôle spécifique. Cette approche vise à minimiser les dépendances entre ces couches, de sorte que les modifications apportées à l'une n'affectent pas les autres. Grâce à cette architecture, les pages web peuvent contenir un minimum de scripts. Voici un schéma qui met en évidence les interactions entre les différentes couches de l'architecture Modèle-Vue-Contrôleur (MVC).

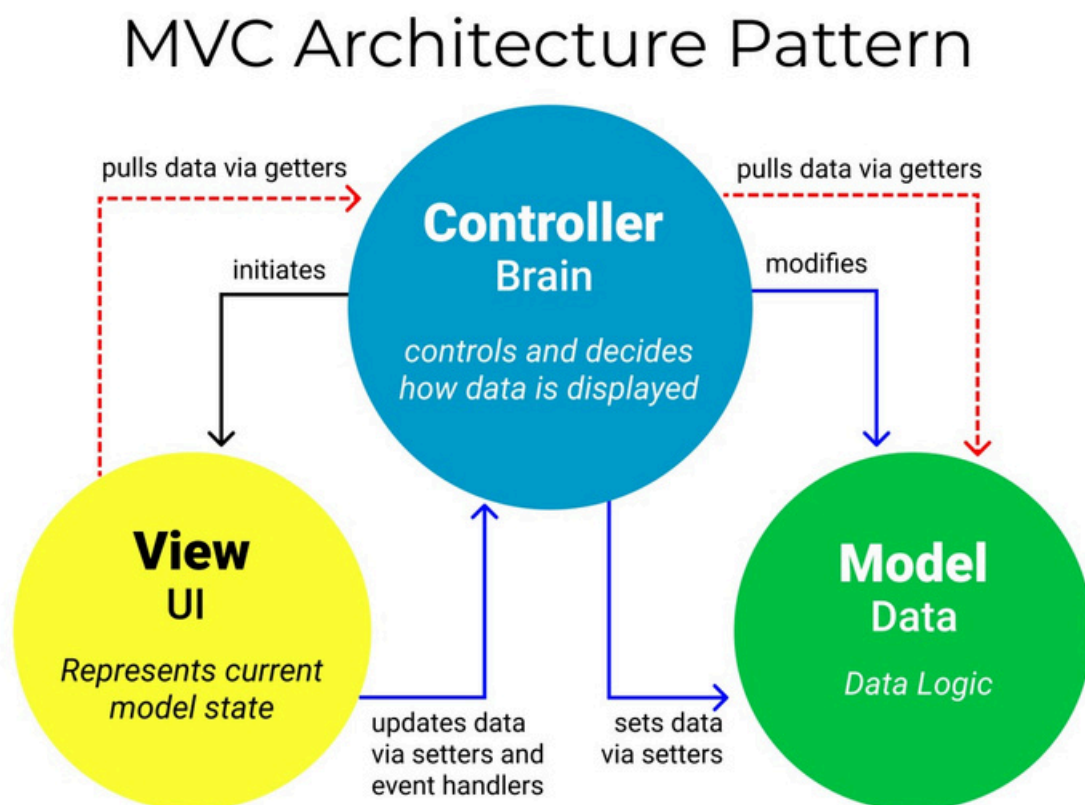


Figure 3 : Schéma d'architecture MVC

Model : Les modèles représentent la structure des données et la logique métier du projet. Ils sont responsables de la gestion des données, qu'elles proviennent de la base de données ou d'autres sources, ainsi que de leur traitement avant qu'elles ne soient utilisées par la vue ou envoyées au contrôleur.

- **Emplacement dans le projet :** Le dossier models contient les différents modèles de données, tels que User, Room, et VerificationCode.

Views : Les vues sont responsables de la présentation des données à l'utilisateur. Elles déterminent l'apparence et l'interactivité de l'interface utilisateur. Dans ce projet, les pages sont responsables de la partie visuelle et de l'affichage des informations.

- **Emplacement dans le projet :** Le dossier pages contient les pages frontend de l'application. Ces pages sont rendues dynamiquement à l'aide de React et Next.js, et elles interagissent avec le contrôleur pour afficher les données et recevoir les entrées utilisateur.

Controlers : Les contrôleurs agissent comme des intermédiaires entre la vue et le modèle. Ils traitent les actions de l'utilisateur, interagissent avec le modèle pour obtenir ou manipuler les données, puis renvoient une réponse à la vue pour l'affichage. Dans ce projet, les contrôleurs sont représentés par les API qui traitent les requêtes HTTP et coordonnent l'accès aux données.

- **Emplacement dans le projet :** Le dossier api contient les routes backend, organisées selon les entités du projet (auth, rooms, users, messages, etc.). Chaque fichier dans api gère des actions spécifiques liées aux données, comme l'ajout d'un utilisateur ou la création d'une room.

III. Outils utilisés

1. Environnement logiciel

- **Visual Studio Code**



Visual Studio Code est un éditeur de code source gratuit et open source développé par Microsoft. Il est connu pour sa légèreté, sa rapidité et ses nombreuses fonctionnalités puissantes telles que l'autocomplétion, la coloration syntaxique, la refactorisation de code, et l'intégration avec Git. Il est également hautement personnalisable grâce à un vaste choix d'extensions disponibles sur son marketplace, et il dispose d'un terminal intégré et d'outils de débogage pour divers langages de programmation.

1. Langages et Technologies utilisés

• Node.js



Node.js est un environnement d'exécution JavaScript côté serveur, basé sur le moteur V8 de Google Chrome. Il permet d'exécuter du JavaScript en dehors du navigateur, ce qui en fait une technologie idéale pour le développement d'applications backend. Node.js repose sur un modèle asynchrone et non-bloquant, ce qui le rend particulièrement adapté aux applications nécessitant une haute performance, comme les systèmes en temps réel, les applications de messagerie instantanée ou les APIs RESTful.

• NPM



npm est un gestionnaire de paquets pour Node.js, permettant aux développeurs d'installer, gérer et partager des bibliothèques et des outils tiers. Il est indispensable dans les projets Node.js, facilitant l'intégration de dépendances, leur gestion de versions et leur mise à jour. Avec npm, les projets peuvent facilement gérer leurs dépendances à travers un fichier package.json.

• React



React est une bibliothèque JavaScript open-source développée par Facebook, permettant de construire des interfaces utilisateur dynamiques. Elle repose sur le concept de composants réutilisables et gère efficacement le DOM avec un virtual DOM. React est utilisé pour le développement du frontend de l'application, garantissant des interfaces réactives et interactives.

• Next.js



Next.js est un framework basé sur React, optimisé pour le rendu côté serveur (SSR - Server-Side Rendering) et la génération de sites statiques. Il permet une gestion fluide du routage, une optimisation automatique des performances, et la possibilité de rendre des pages avant même qu'elles ne soient envoyées au navigateur, ce qui améliore la performance et l'optimisation pour le SEO.

Next.js REST API :

Next.js fournit un moyen intégré pour créer des API REST directement dans le même projet. Cela permet de définir des routes API au sein du même dossier que les pages,

• JavaScript



JavaScript est un langage de programmation largement utilisé pour créer des pages web interactives. C'est le langage de base pour la manipulation de la structure du DOM (Document Object Model) des pages web, permettant de rendre l'interface dynamique et réactive aux actions de l'utilisateur. Utilisé côté client (dans le navigateur) mais aussi côté serveur grâce à Node.js, JavaScript est au cœur du développement web moderne.

• CSS



Le CSS (Cascading Style Sheets) est un langage utilisé pour décrire la présentation des documents HTML. Il permet de styliser les éléments HTML en définissant des propriétés telles que les couleurs, la disposition, la typographie et les animations. En combinaison avec HTML et JavaScript, CSS permet de créer des interfaces visuellement attrayantes et réactives.

• Tailwind CSS



Tailwind CSS est un framework CSS utilitaire qui permet de créer rapidement des interfaces personnalisées sans avoir à écrire des classes CSS complexes. Avec une approche basée sur des classes utilitaires prédéfinies, Tailwind simplifie la gestion des styles et accélère le processus de développement tout en offrant une grande flexibilité.

• GSAP



GSAP (GreenSock Animation Platform) est une bibliothèque JavaScript puissante pour les animations complexes et performantes. Elle permet de créer des animations fluides et interactives avec un contrôle précis sur la timeline.

• Animate.css



animate.css est une bibliothèque d'animations CSS prêtes à l'emploi, permettant d'ajouter des effets d'animation rapides et simples en utilisant des classes prédéfinies

• MongoDB



MongoDB est une base de données NoSQL orientée document qui permet de stocker des données dans un format flexible, utilisant des documents BSON (une forme binaire de JSON). Elle est idéale pour des applications qui requièrent une structure de données évolutive et scalabilité horizontale.

Mongoose :

Mongoose est un ODM (Object Data Modeling) pour MongoDB en Node.js. Il simplifie la gestion des données MongoDB en fournissant une syntaxe basée sur des modèles, assurant la validation des données et l'exécution des requêtes.

• Axios



Axios est une bibliothèque JavaScript permettant de faire des requêtes HTTP depuis le frontend ou le backend. Elle est utilisée pour faciliter la communication serveur-client, permettant d'envoyer et de recevoir des données de manière simple avec une syntaxe moderne.

• Next-auth



Next-auth est une bibliothèque d'authentification pour Next.js, permettant de mettre en place une gestion sécurisée des utilisateurs avec un grand nombre de fournisseurs d'authentification.

MongoDB-adapter :

MongoDB-adapter est un adaptateur pour stocker les sessions et utilisateurs dans MongoDB, facilitant l'intégration de Next-auth avec MongoDB.

• Bcrypt.js



Bcrypt.js est une bibliothèque utilisée pour le hachage des mots de passe en JavaScript. Elle permet de sécuriser les mots de passe des utilisateurs en les convertissant en une forme chiffrée (hashée), rendant ainsi les informations sensibles beaucoup plus difficiles à compromettre.

• jsonwebtoken



jsonwebtoken (JWT) est une bibliothèque permettant de générer et vérifier des JSON Web Tokens pour la gestion des sessions et de l'authentification.

• Cookie.js



Cookie.js est une bibliothèque pour manipuler facilement les cookies dans le navigateur, facilitant leur gestion et leur lecture. Elle est utilisée dans notre projet pour gérer les sessions.

Cookie-parser :

cookie-parser est un middleware pour Express.js permettant de lire et d'analyser les cookies dans les requêtes HTTP.

• Google-auth-library



C'est une bibliothèque qui permet d'intégrer l'authentification Google dans une application. Elle facilite la gestion de l'authentification avec des comptes Google pour offrir une solution de login simplifiée.

Google One-Tap :

Google One-Tap est un système de connexion rapide utilisant Google, permettant aux utilisateurs de se connecter en un seul clic, sans avoir à saisir leur mot de passe.

• SendGrid



SendGrid/Mail est un service de messagerie cloud permettant d'envoyer des emails à grande échelle. Il offre des outils puissants pour la gestion des emails transactionnels et des notifications.

Nodemailer :

Nodemailer est une bibliothèque Node.js permettant d'envoyer des emails depuis une application. Elle supporte divers services de messagerie et permet d'envoyer des emails de manière simple et sécurisée.

• Socket.io



Socket.io est une bibliothèque permettant de gérer la communication en temps réel entre le client et le serveur via des websockets. Elle est utilisée dans ce projet pour créer un système de messagerie instantanée, permettant aux utilisateurs d'envoyer et de recevoir des messages en temps réel dans les différentes rooms.

Chapitre 3 : Réalisation de l'application

I. Introduction

Dans la phase de réalisation, je présente mon application à travers un ensemble d'interfaces. Cette présentation permettra de donner un aperçu visuel de ma solution et de comprendre les fonctionnalités clés de l'application.

II. Logo du Site Web



Le nom du projet, "**OBSIDIAN**", a été choisi en référence à l'obsidienne, une pierre volcanique naturelle connue pour sa couleur noire profonde et ses propriétés uniques. Ce matériau est souvent associé à la pureté, la force, et la transparence. Dans le contexte de ce projet, le choix de ce nom reflète l'objectif de créer une plateforme web à la fois solide, moderne et claire. L'obsidienne est également perçue comme une pierre précieuse, qui

symbolise la recherche de l'excellence et de la qualité, des valeurs que l'on souhaite intégrer dans la conception et le développement de l'application.. Dans un autre contexte, l'obsidienne représente un portail vers une expérience nouvelle et inédite pour les utilisateurs, tout comme dans le jeu vidéo Minecraft, où l'obsidienne sert à créer un portail qui transporte les joueurs vers un autre monde. Ce parallèle illustre parfaitement l'idée que ce site web agira comme un passage vers un univers numérique différent, où l'utilisateur pourra interagir et explorer des espaces virtuels uniques. Ce logo a été choisi pour souligner l'innovation et l'aspect immersif du projet, tout en créant un lien symbolique fort entre la plateforme et les possibilités infinies qu'elle offre.

III. Présentation des interfaces de l'application

1. Interface d'accueil

La page d'accueil sert de point d'entrée principal pour les utilisateurs. Elle affiche les rooms auxquelles l'utilisateur a déjà rejoint, offrant ainsi une vue d'ensemble rapide de ses espaces de discussion. L'interface est simple, mais dynamique, avec un design intuitif qui permet à l'utilisateur de naviguer facilement entre les rooms et d'interagir avec les autres membres. La page inclut également des éléments interactifs, comme des notifications ou des alertes concernant les nouvelles rooms ou les messages reçus.

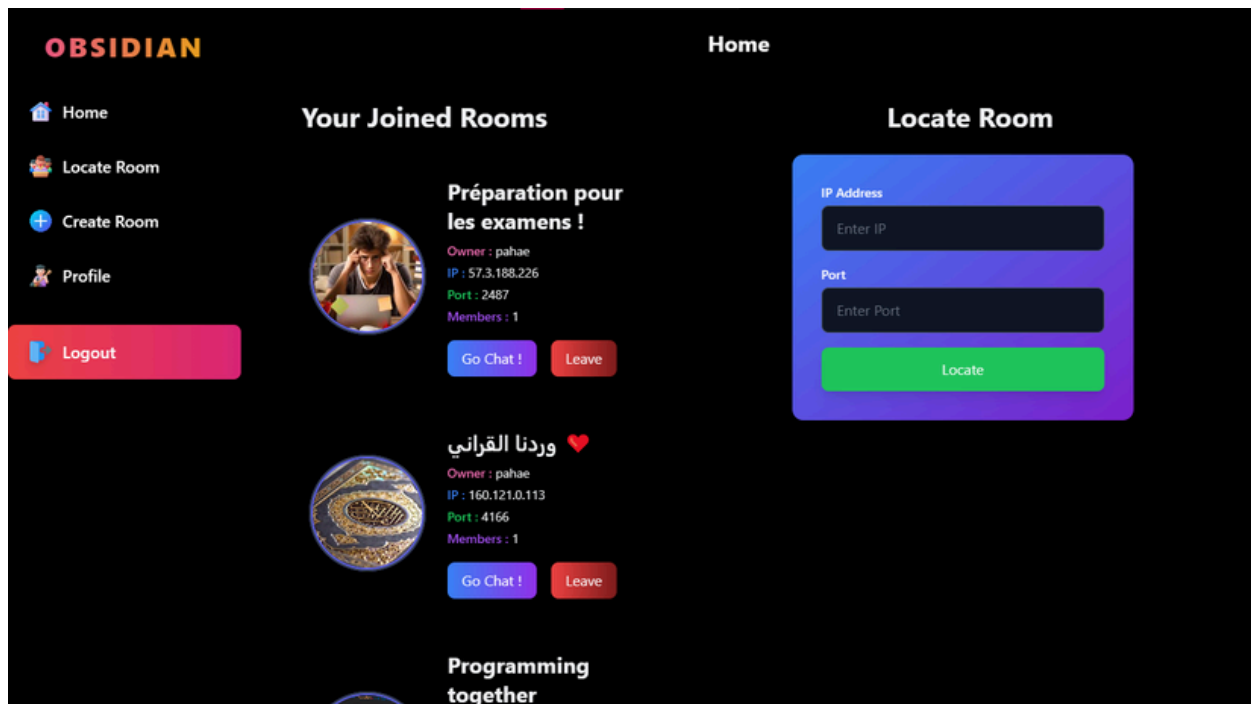
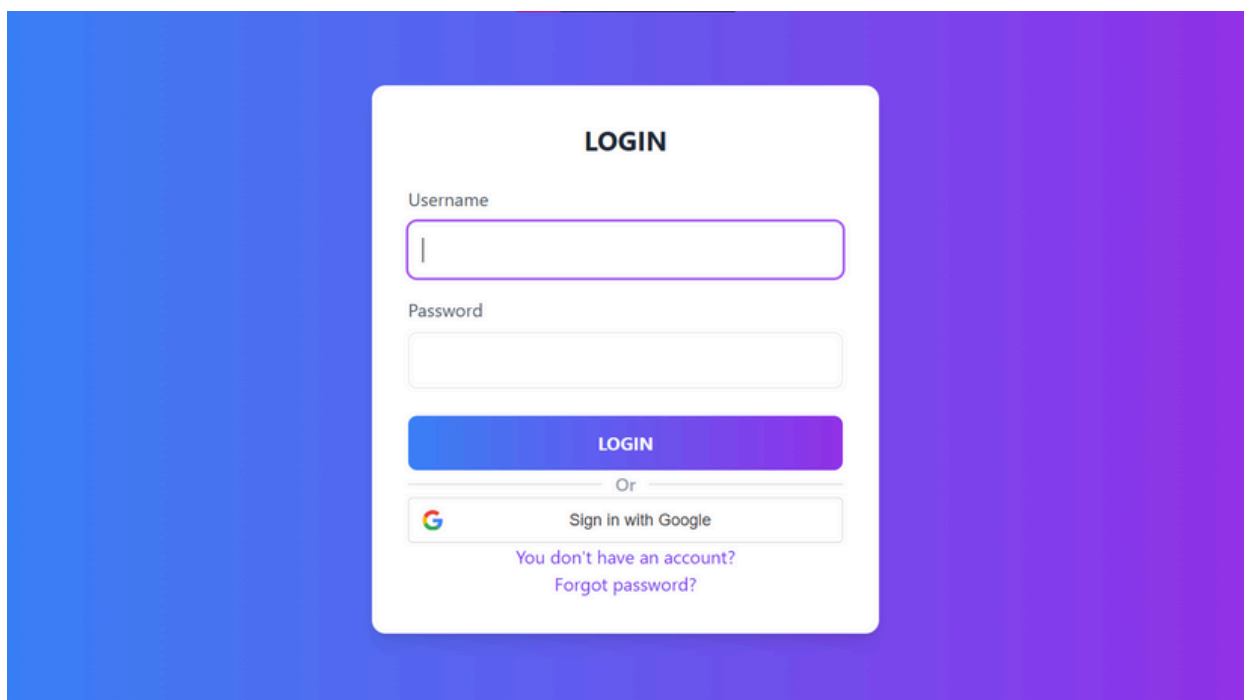


Figure 4 : Page d'accueil

1. Page de connexion

Cette page permet à l'utilisateur de se connecter à son compte. Il peut choisir de se connecter soit avec son nom d'utilisateur et son mot de passe classiques, soit via l'authentification Google.

On ne peut accéder à aucune page dans le site via son lien sans avoir être connecté d'abord.



Des messages d'erreur ou des instructions sont affichés en cas de problème (par exemple, mot de passe incorrect ou compte inexistant).

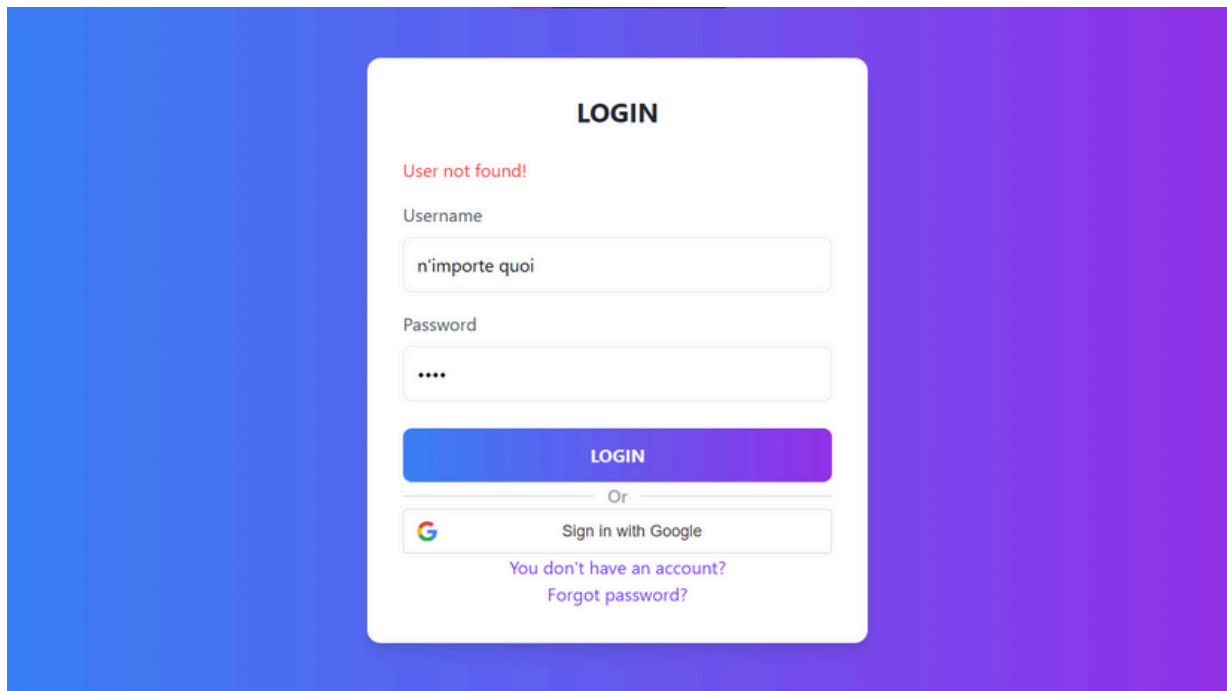


Figure 5 : Page de connexion «message d'erreur»

2. Page de registration

La page d'enregistrement est dédiée à la création d'un nouveau compte. Elle est interactive, avec des étapes guidées pour remplir les informations nécessaires comme le nom, l'email, et le mot de passe. L'animation de cette page permet une expérience utilisateur fluide et agréable.

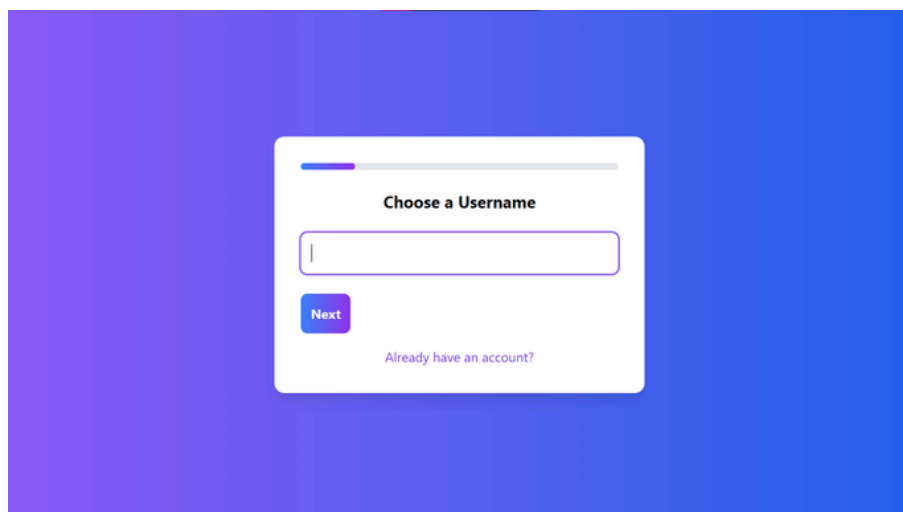


Figure 6 : Page de registration «choix d'un username»

On aura ici un champ pour l'email, qui sera utilisé pour la vérification du compte, la récupération du mot de passe, et la communication avec l'utilisateur, et un champ pour saisir le mot de passe. Ce champ doit être sécurisé et inclure une vérification de la force du mot de passe (par exemple, minimum de 8 caractères, inclusion de chiffres et de lettres, etc.).

The image shows a registration form titled "Enter Your Email" centered on a blue gradient background. The form is a white rounded rectangle with a progress bar at the top, where the first step is highlighted in blue. Below the title is a text input field for the email address. At the bottom of the form are two buttons: a grey "Previous" button on the left and a blue "Next" button on the right. Below the buttons is a link that says "Already have an account?" in a smaller, lighter blue font.

Figure 7 : Page de registration «champ de l'email»

The image shows a registration form titled "Choose a Password" centered on a blue gradient background. The form is a white rounded rectangle with a progress bar at the top, where the second step is highlighted in blue. Below the title is a text input field for the password. At the bottom of the form are two buttons: a grey "Previous" button on the left and a blue "Next" button on the right. Below the buttons is a link that says "Already have an account?" in a smaller, lighter blue font.

Figure 8 : Page de registration «champ du mot de passe»

- Lors de la saisie, il y a une validation en temps réel pour chaque champ, avec des messages d'erreur indiquant si le champ est mal rempli (par exemple, email invalide, mot de passe trop faible).
- Un bouton "Suivant" est activé uniquement lorsque tous les champs sont remplis correctement.

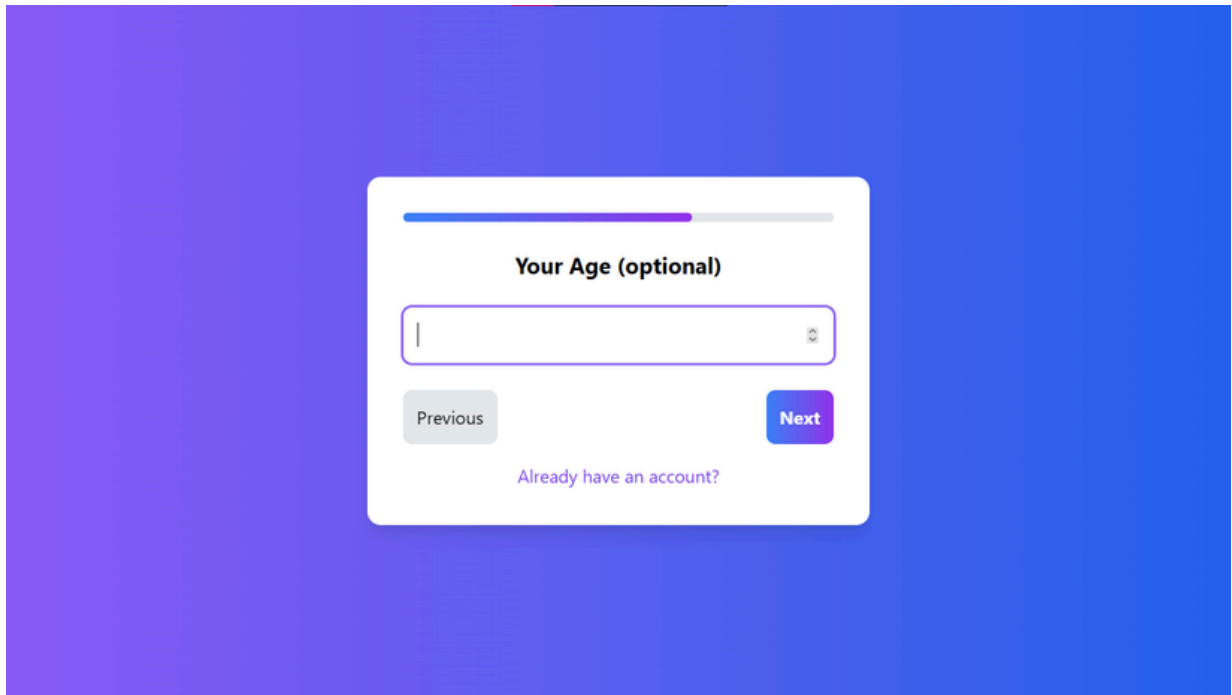
A screenshot of a registration form titled "Your Age (optional)". The form is centered on a blue gradient background. It features a progress bar at the top, a text input field for age, a "Previous" button, a "Next" button, and a link "Already have an account?".

Figure 9 : Page de registration «champ de l'âge»

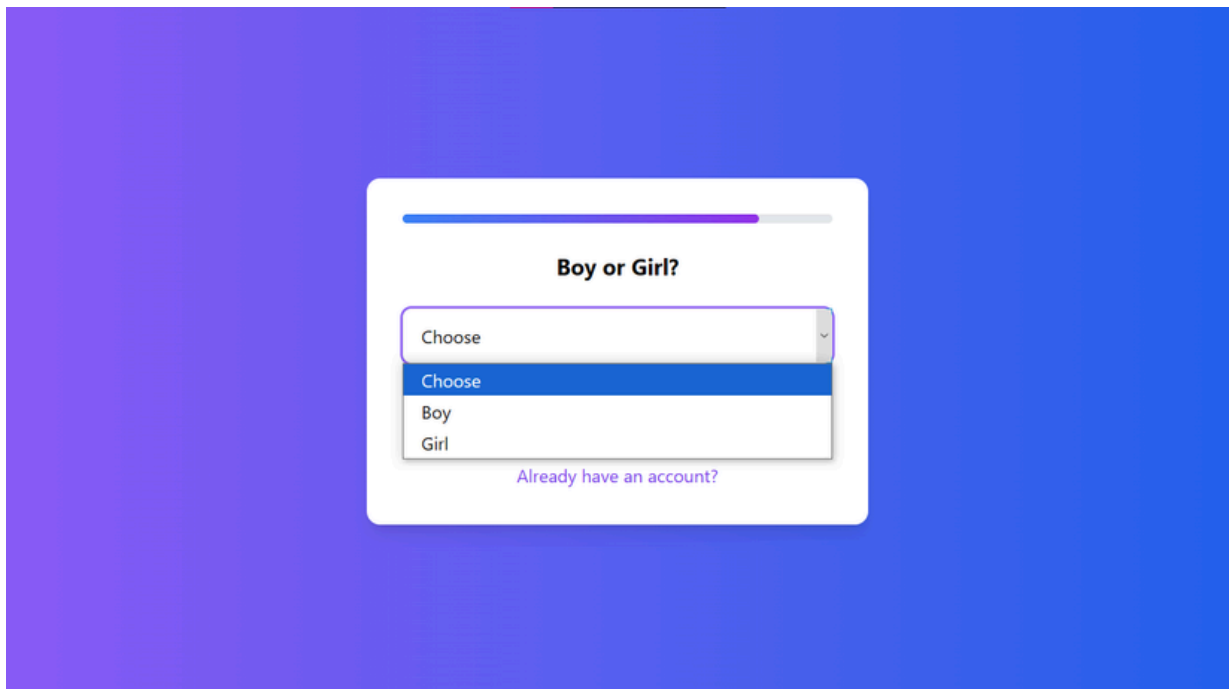
A screenshot of a registration form titled "Boy or Girl?". The form is centered on a blue gradient background. It features a progress bar at the top, a dropdown menu with options "Choose", "Boy", and "Girl", a "Previous" button, a "Next" button, and a link "Already have an account?".

Figure 10 : Page de registration «champ du sex»

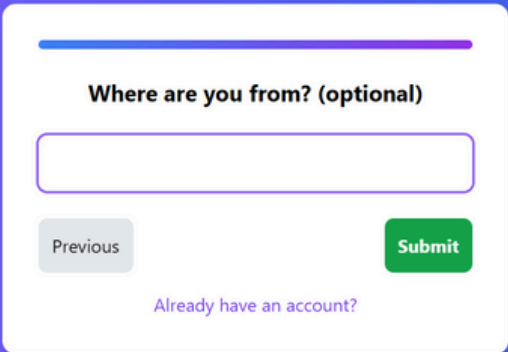
A registration form titled "Where are you from? (optional)" is displayed on a blue gradient background. The form has a white background and a rounded rectangle shape. It features a text input field with a purple border. Below the input field are two buttons: a grey "Previous" button on the left and a green "Submit" button on the right. At the bottom of the form, there is a link that says "Already have an account?" in purple text. A progress bar at the top of the form shows a blue segment followed by a purple segment.

Figure 11 : Page de registration «champ de nationalité»

Ces étapes facultatives peuvent inclure une validation de l'âge ou du sexe selon les restrictions de certaines rooms, et les données sont vérifiées avant la validation. Si toutes les informations sont correctes, une notification de succès sera affichée.

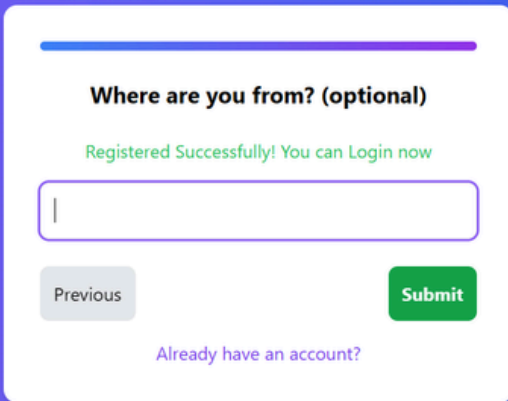
A registration form titled "Where are you from? (optional)" is displayed on a blue gradient background. The form has a white background and a rounded rectangle shape. It features a text input field with a purple border. Below the input field are two buttons: a grey "Previous" button on the left and a green "Submit" button on the right. At the bottom of the form, there is a link that says "Already have an account?" in purple text. A progress bar at the top of the form shows a blue segment followed by a purple segment. A green message "Registered Successfully! You can Login now" is displayed above the input field.

Figure 12 : Page de registration «succès»

3. Page de Récupération du Mot de Passe Oublié

(Étape 1) : Demande de Récupération

- **Objectif** : Demander à l'utilisateur de saisir l'email associé à son compte pour initier le processus de récupération de mot de passe.
- **Champs à remplir** : Un champ de saisie dans lequel l'utilisateur entre l'email lié à son compte sur le site.
- **Validation** : L'email doit être validé en temps réel pour s'assurer qu'il respecte le format standard (ex. : utilisateur@example.com), si l'email n'est pas valide ou si l'email fourni n'est pas associé à un compte existant, un message d'erreur s'affiche, demandant à l'utilisateur de vérifier l'email ou de s'assurer qu'il est inscrit.
- **Bouton** : Un bouton "Send code" qui devient activé seulement lorsque l'email est valide. Une fois l'email validé, envoie une requête au serveur pour générer un code de vérification

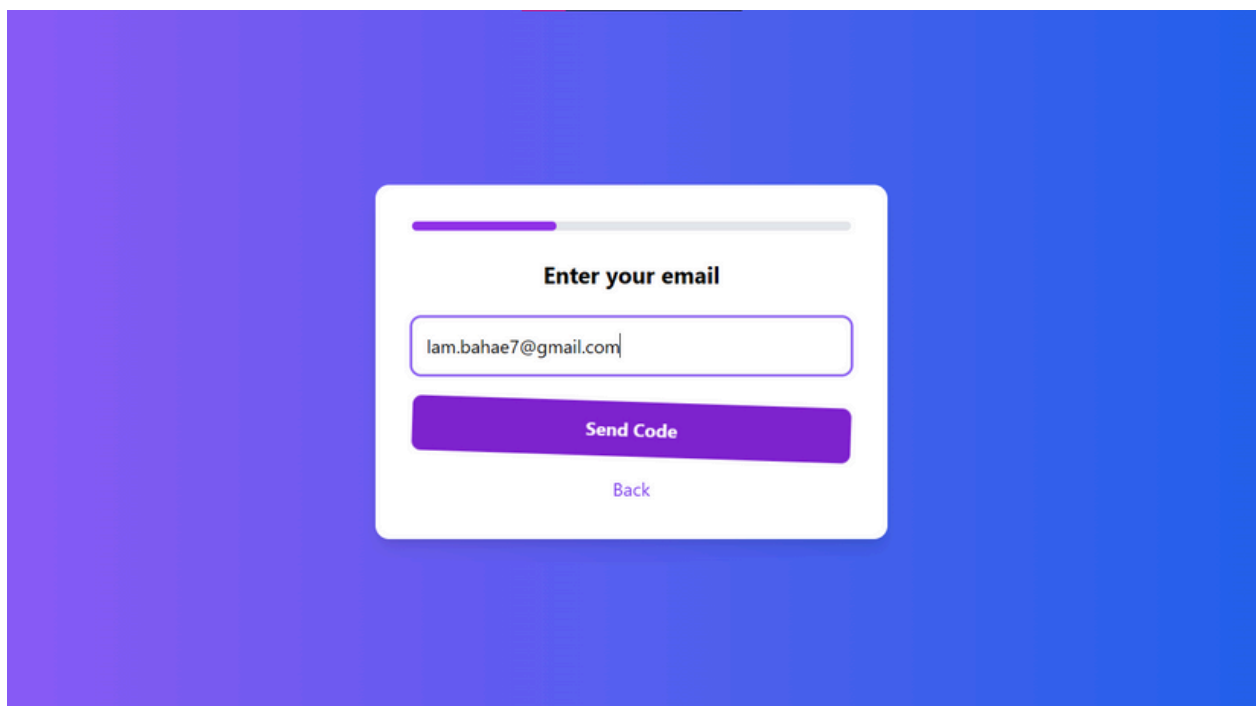
The image shows a web interface for email recovery. It features a white modal box centered on a blue-to-purple gradient background. At the top of the modal is a progress bar with the first segment highlighted in purple. Below the progress bar is the heading "Enter your email". Underneath is a text input field containing the email address "lam.bahae7@gmail.com". Below the input field is a large purple button labeled "Send Code". At the bottom of the modal is a smaller, light blue link labeled "Back".

Figure 13 : Page de récupération du compte «saisir l'email»

(Étape 2) : Envoi du Code de Vérification

- **Objectif** : Envoyer un code de vérification temporaire à l'email de l'utilisateur pour garantir qu'il est bien le propriétaire du compte.
- **Fonctionnement** : Après que l'utilisateur ait soumis son email, le système génère un code de vérification aléatoire de 6 chiffres et l'envoie à l'adresse email fournie, ce code est valable pendant 5 minutes. Cette limite de temps garantit la sécurité du processus et empêche un usage abusif du lien.
- **Message de confirmation** : Une fois le code envoyé, un message informatif s'affiche pour informer l'utilisateur que le code a été envoyé à son email. Ce message peut contenir un avertissement indiquant que le code expirera dans 5 minutes.
- **Bouton** : Un bouton "Confirm code" qui envoie une requête au serveur pour valider le code de vérification

L'adresse e-mail existante reçoit un code de vérification à 6 chiffres pour validation. L'utilisateur le saisit sur une page dédiée, avec affichage d'un message d'erreur en cas de saisie incorrecte.

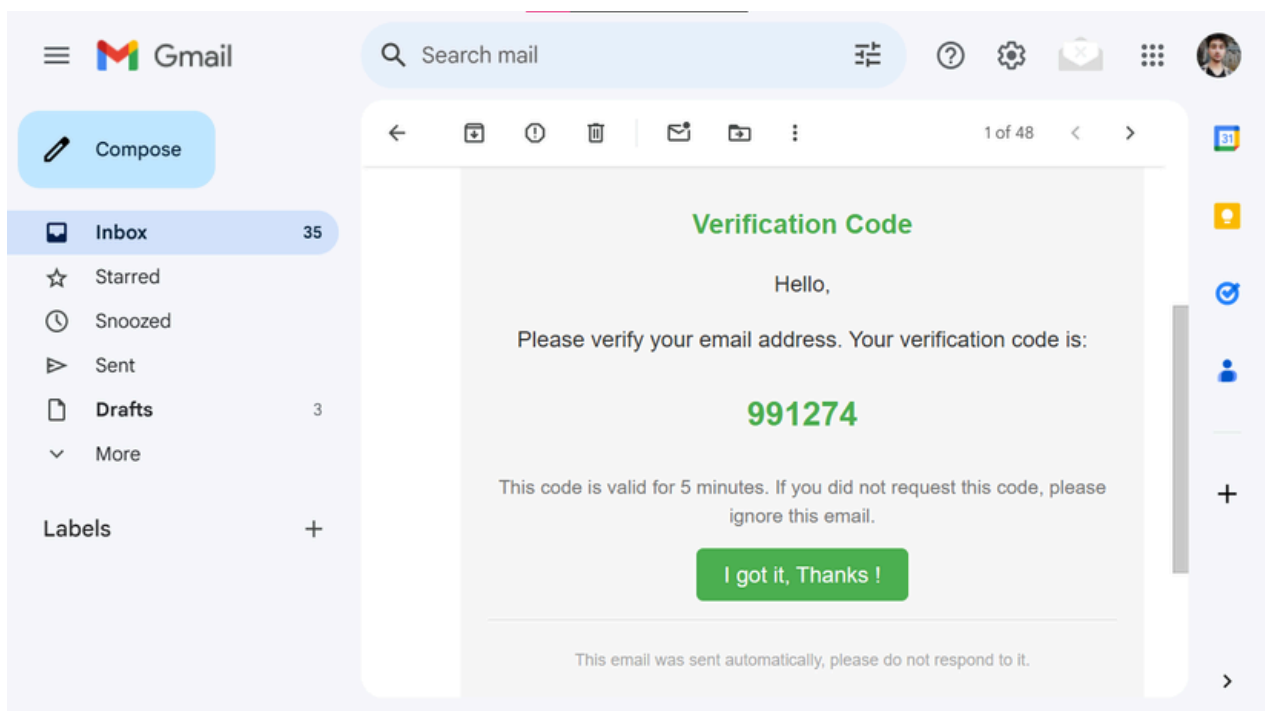


Figure 14 : Page de récupération du compte «code reçu sur gmail»

(Étape 3) : Validation du Code de Vérification

- **Objectif** : Permettre à l'utilisateur de saisir le code de vérification reçu par email pour valider sa demande de réinitialisation du mot de passe.
- **Validation** : Lors de la saisie, le code est vérifié en temps réel côté serveur, si le code est invalide ou expiré (plus de 5 minutes), un message d'erreur s'affiche, informant l'utilisateur que le code est incorrect ou périmé.

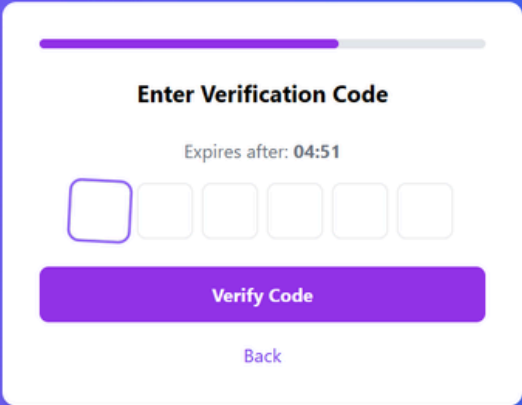


Figure 15 : Page de récupération du compte «champ du code de vérification»

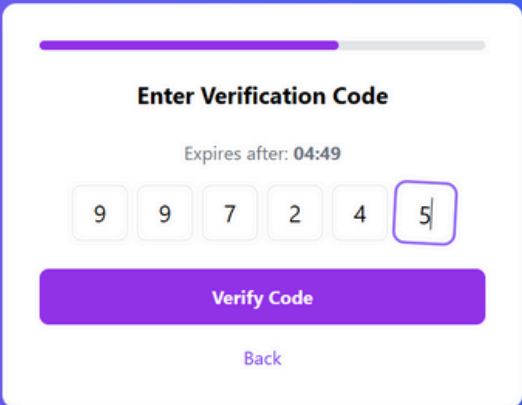


Figure 16 : Page de récupération du compte «saisi de code de vérification»

(Étape 4) : Réinitialisation du Mot de Passe

- **Objectif** : Permettre à l'utilisateur de saisir un nouveau mot de passe sécurisé une fois que le code de vérification a été validé.
- **Champs à remplir** : Un champ où l'utilisateur saisit son nouveau mot de passe, et un deuxième pour confirmer que le mot de passe a bien été saisi correctement.
- **Validation** : Un message d'erreur s'affiche si les deux mots de passe saisis ne correspondent pas. Une fois que l'utilisateur a validé son nouveau mot de passe, un message de confirmation apparaît pour lui indiquer que son mot de passe a été réinitialisé avec succès. Un bouton "Back" ou "Retour à la page de connexion" lui permet d'accéder à la page de connexion pour utiliser son nouveau mot de passe.
- **Bouton** : Le bouton "Save" devient activé lorsque l'utilisateur remplit correctement les champs du nouveau mot de passe et de confirmation. En cliquant dessus, une requête est envoyée au serveur pour mettre à jour le mot de passe après validation du code de vérification. Si l'opération est réussie, l'utilisateur reçoit une confirmation et est redirigé vers la page de connexion. Ce processus assure la sécurité de la réinitialisation en validant à la fois le code et la correspondance des mots de passe.

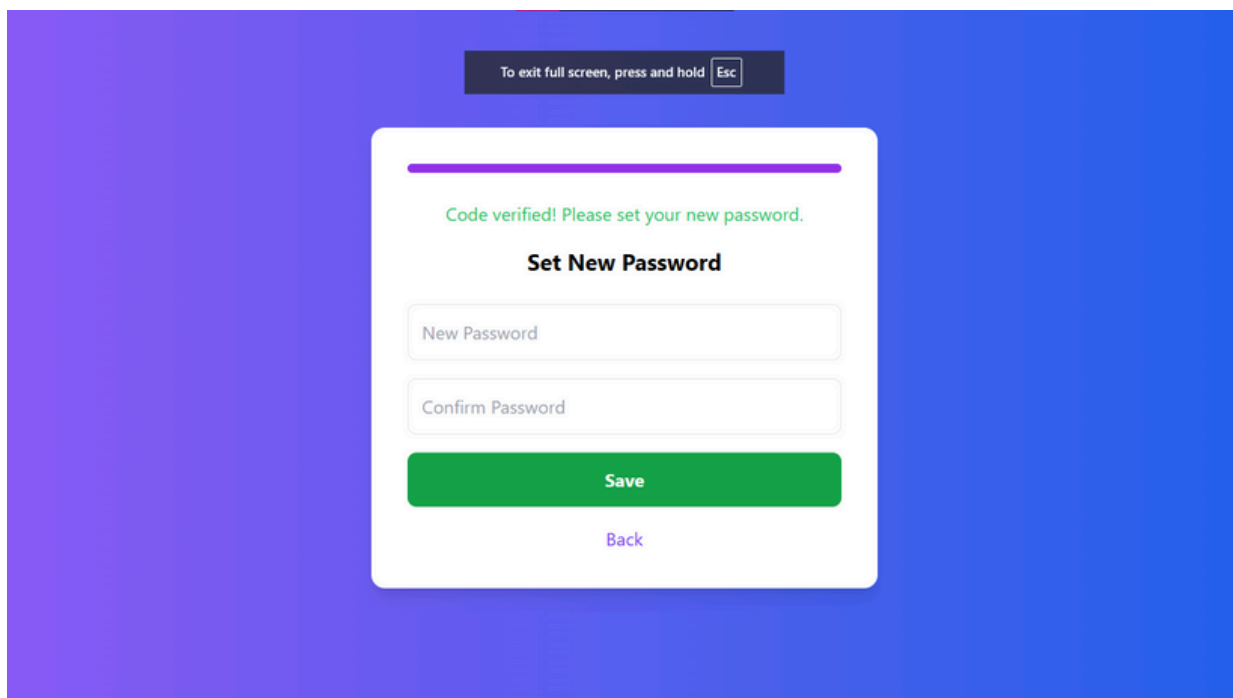


Figure 17 : Page de récupération du compte «saisi de code de vérification»

3. Page de connexion avec Google

Le login avec Google permet à l'utilisateur de se connecter rapidement en utilisant son compte Google, simplifiant ainsi l'authentification sans avoir à mémoriser un mot de passe supplémentaire. Lors de la connexion, l'utilisateur clique sur le bouton "Se connecter avec Google", ce qui le redirige vers une fenêtre de validation de son compte Google. Une fois l'autorisation accordée, l'application récupère les informations de base de l'utilisateur (comme l'email) et ici on va traiter deux cas.

- **Cas 1** : Si ce compte n'est pas encore enregistré dans la base de données (comme connexion normale), un compte sera automatiquement créé et enregistré
- **Cas 2** : Si l'utilisateur possède déjà un compte lié à cet email, on va tout simplement lui connecter avec ce compte, la session est immédiatement établie.

Cette méthode d'authentification offre une expérience utilisateur fluide et sécurisée, tout en réduisant les risques liés aux mots de passe.

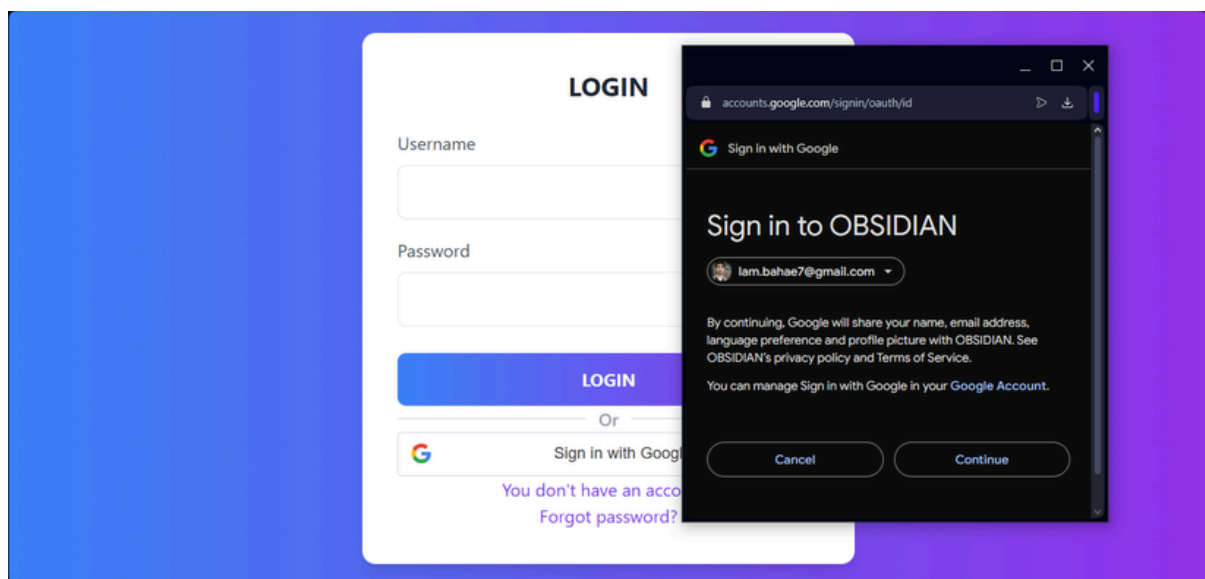


Figure 18 : Page de récupération du compte «saisi de code de vérification»

3. Page de profile

La page Profile est une fonctionnalité clé du site, offrant à chaque utilisateur la possibilité de gérer ses informations personnelles et de consulter les rooms qu'il a créées ou rejoint. Lorsqu'un utilisateur accède à cette page, il peut voir un aperçu détaillé de ses données de profil, telles que son nom d'utilisateur, son adresse email, son âge, son sexe, sa bio, ainsi que l'image de profil qu'il a choisie.

Section d'informations personnelles :

- Cette section affiche les informations de base de l'utilisateur, telles que son nom, email, âge et sexe. L'utilisateur peut ici modifier ces données en fonction de ses préférences.
- Il a également la possibilité de télécharger ou de changer l'image de profil pour personnaliser davantage son compte.

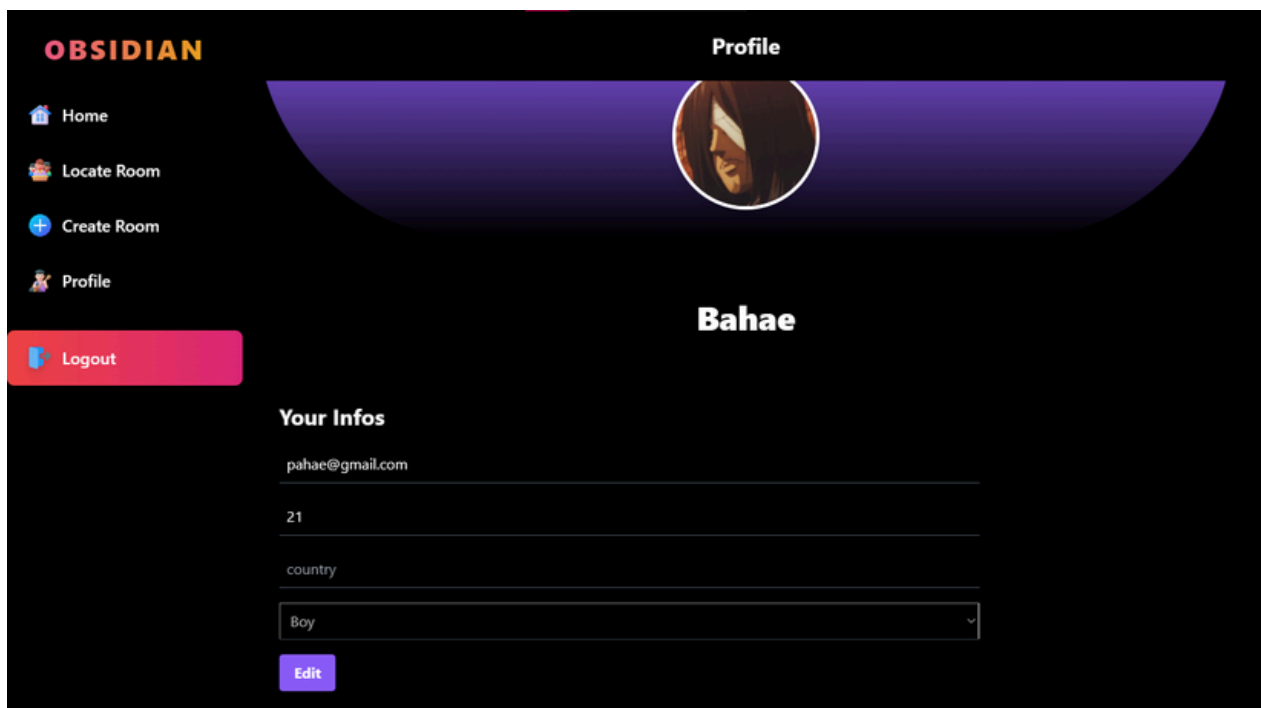


Figure 19 : Page de profile «informations personnelles»

Modifier les informations :

- Un bouton "Modifier" permet à l'utilisateur de mettre à jour ses informations personnelles. En cliquant sur ce bouton, un formulaire apparaît où l'utilisateur peut modifier ses données. Ces informations sont ensuite mises à jour dans la base de données une fois validées.
- Les images se stockent comme un String condensée de données codées en base64

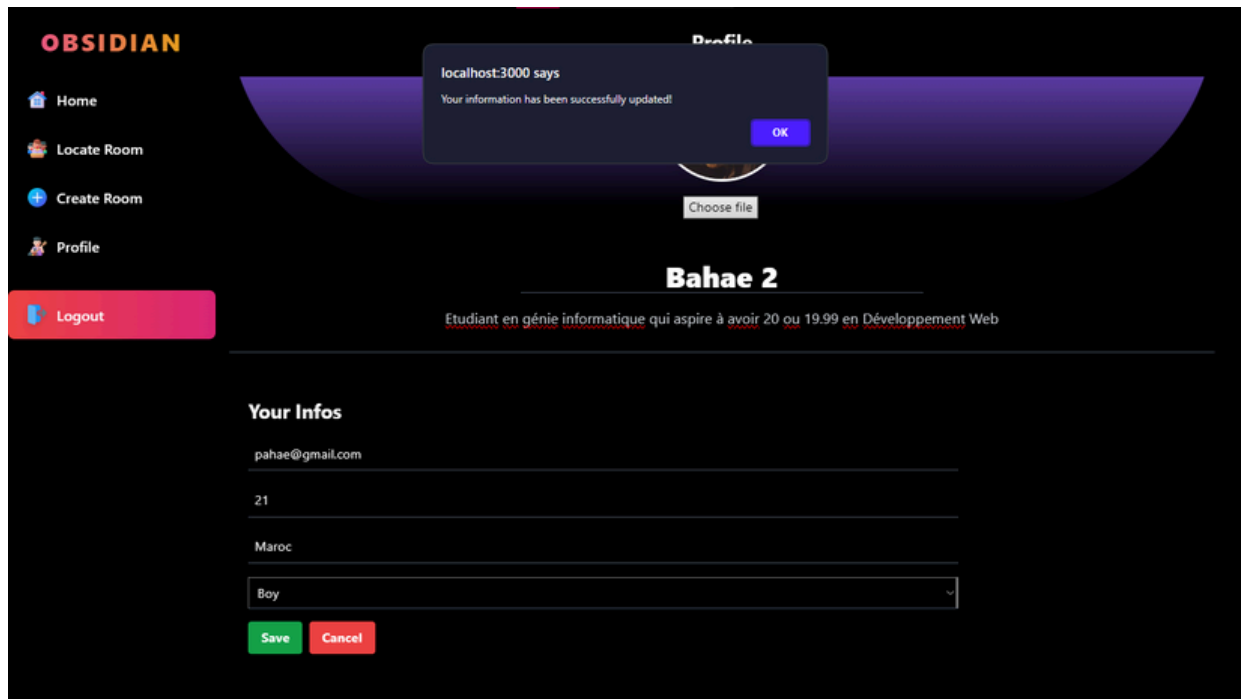


Figure 20 : Page de profile «modification des informations»

Liste des rooms créées :

- Cette section affiche toutes les rooms que l'utilisateur a créé. Pour chaque room, des informations telles que le nom, l'IP, le port, le nombre de membres, et une miniature de l'image associée à la room sont visibles. L'utilisateur peut interagir avec chaque room, soit pour voir plus de détails, soit pour quitter la room.

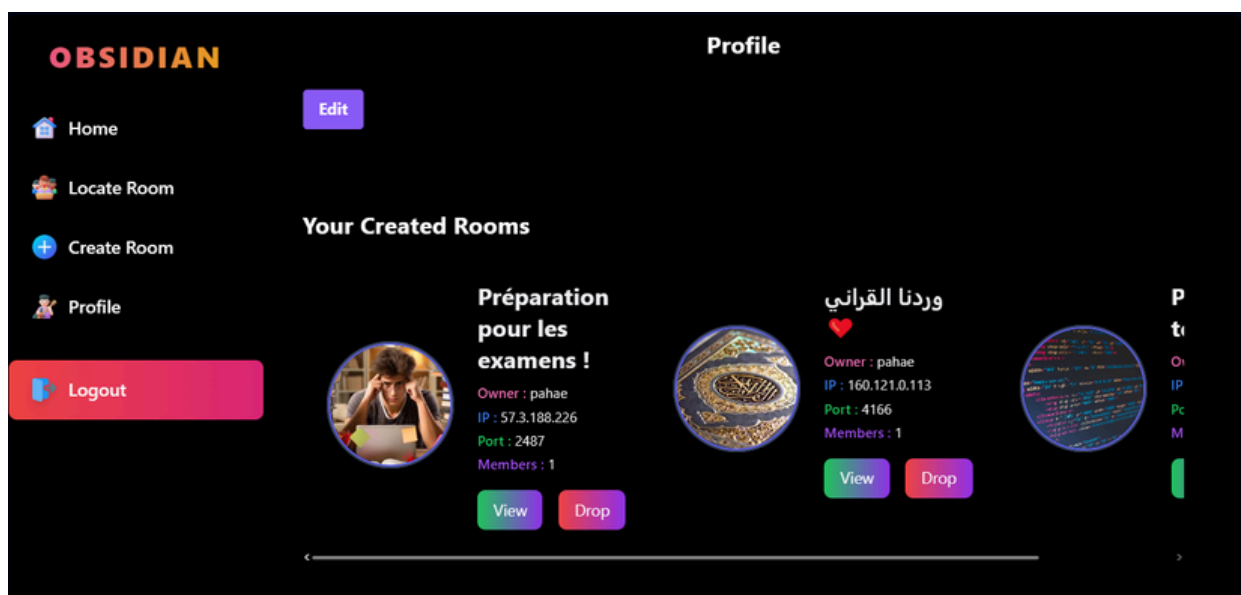


Figure 21 : Page de profile «modification des informations»

Suppression d'une room :

- En cliquant sur "Drop" on peut supprimer définitivement la room
- Une requête HTTP sera envoyé pour le backend pour supprimer la room de la base de données, puis la supprimer de la liste des rooms jointées par chaque utilisateur.

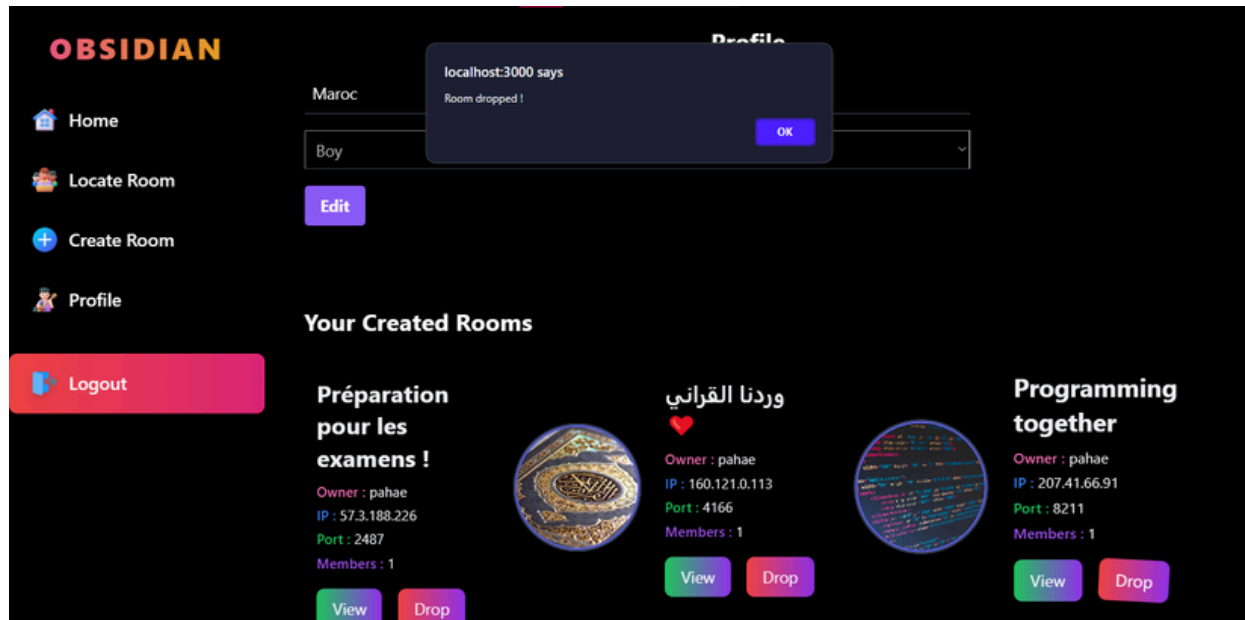


Figure 22 : Page de profile «modification des informations»

Suppression du compte utilisateur :

- Bouton de suppression :
 - En bas de la page, un bouton "Delete my account permanently" est visible, permettant à l'utilisateur de supprimer définitivement son compte.
- Processus de suppression :
 - Une fois confirmé :
 - Toutes les données liées à l'utilisateur (informations personnelles, rooms créées, messages envoyés, etc.) sont supprimées de manière permanente de la base de données.
 - Les rooms créées par l'utilisateur ne seront pas supprimées, et les membres de ces rooms peuvent
 - Pour chaque room jointée par cet utilisateur, le nombre des membres sera décrémenté.
 - Message final :
 - Après suppression, la session est détruite, et l'utilisateur est redirigé vers la page d'accueil du site.

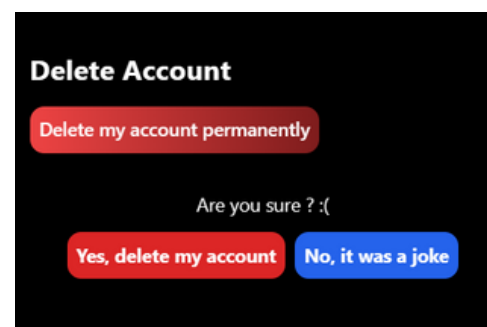


Figure 23 : Page de profile «suppression du compte»

3. Page de creation d'une room

Sur cette page, l'utilisateur peut créer une nouvelle room de discussion. Un formulaire permet de définir le nom, la description, les paramètres (tels que les restrictions d'âge et de sexe, le nombre de membres maximum, etc.), et l'image associée à la room (peut être animée en téléchargeant l'image sous format GIF). Une fois la room créée, l'utilisateur y est automatiquement ajouté.

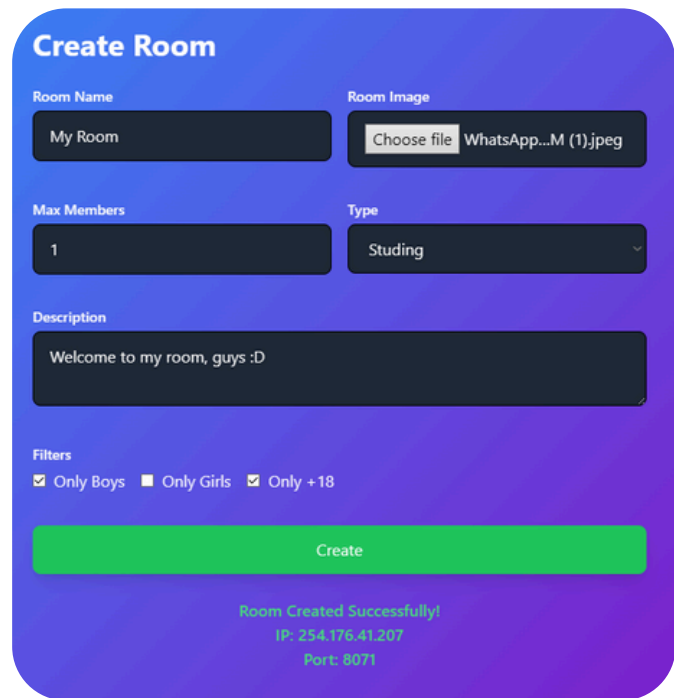


Figure 24 : Page de création d'une room

4. Page de recherche d'une room

Cette page permet à l'utilisateur de rechercher une room en fonction de son IP et de son PORT. Un moteur de recherche rapide et performant permet d'afficher les résultats pertinents en temps réel. Cette page facilite également l'accès aux rooms, tout en vérifiant si l'utilisateur satisfait aux restrictions des rooms (âge, sexe, nombre de membres, etc.).

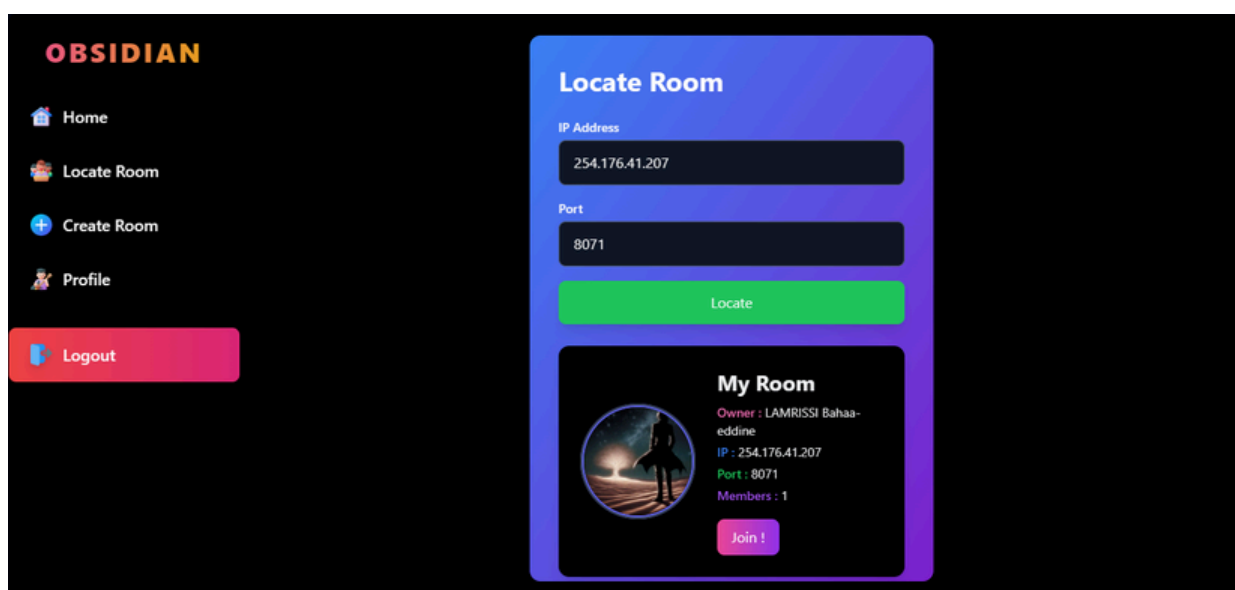
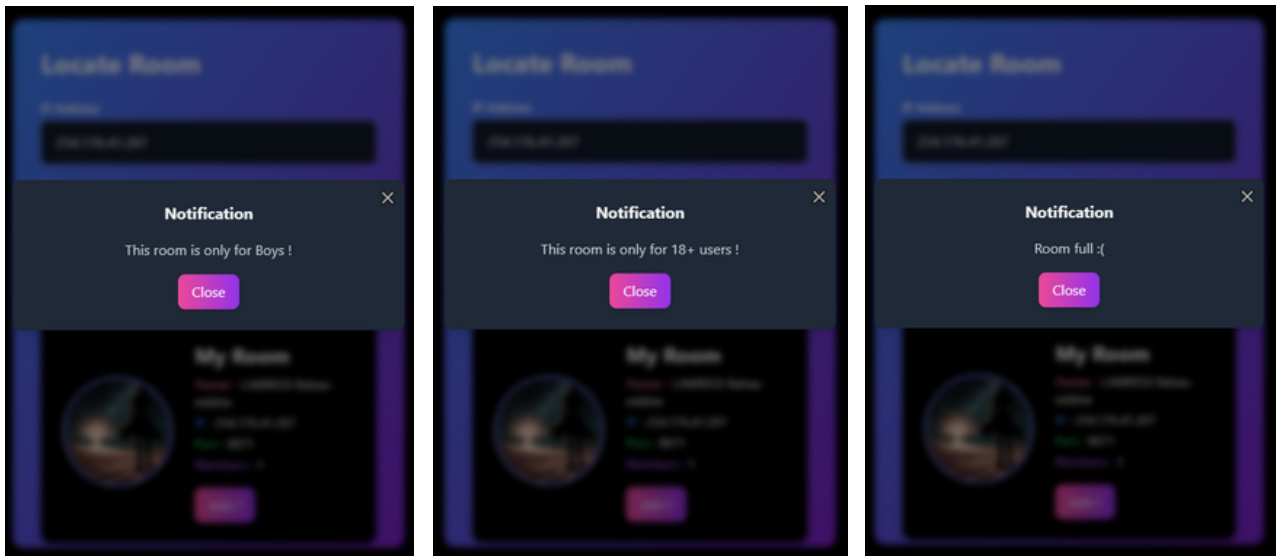


Figure 25 : Page de recherche d'une room

Si l'utilisateur n'obéit pas à certaine restriction ou le nombre maximal d'utilisateurs est atteint pour cette room, il ne pourra pas la joindre, et un message sera affiché pour lui expliquer la raison qu'elle lui a empêché de joindre la room. Ainsi, un utilisateur ne peut pas rejoindre une room déjà rejointe par lui.



Figures 26, 27, 28 : Page de recherche d'une room «restrictions»

4. Page de recherche d'une room

Cette page permet à l'utilisateur de rechercher une room en fonction de son IP et de son PORT. Un moteur de recherche rapide et performant permet d'afficher les résultats pertinents en temps réel. Cette page facilite également l'accès aux rooms, tout en vérifiant si l'utilisateur satisfait aux restrictions des rooms (âge, sexe, nombre de membres, etc.).

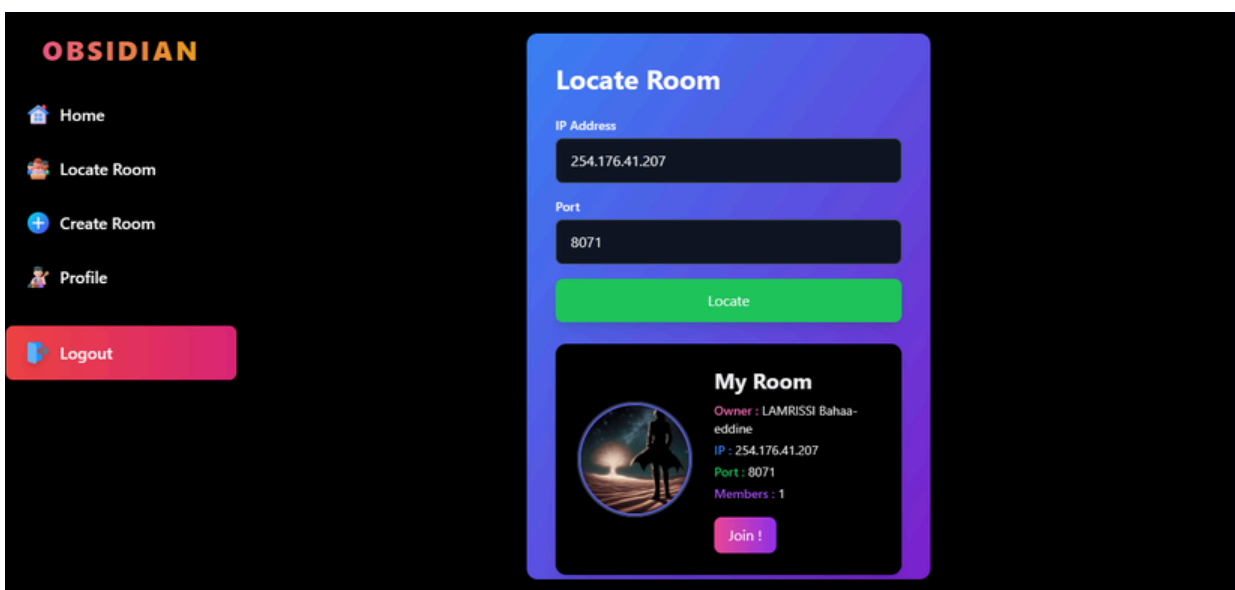


Figure 29 : Page de recherche d'une room

5. Page de chat dans une room

La page Room est une page dynamique qui affiche les détails d'une room spécifique à laquelle un utilisateur a accès à partir de l'id passé en lien de de la page (*exemple : http://localhost/Room?id=id_d'une_room*). Elle permet à l'utilisateur de participer activement à une room en envoyant des messages, en visualisant les messages envoyés, et en interagissant avec d'autres membres de la room, les messages s'affichent en temps réel grâce à Socket.io, une bibliothèque permettant d'assurer la communication bidirectionnelle entre le client et le serveur sans avoir besoin de rafraîchir la page.

Démonstration :

Dans cet exemple j'ai ouvert deux comptes dans deux différents navigateurs, le premier compte avec un username "**Bahae 2**" et le deuxième avec "**Rayan**", et j'ai envoyé des messages d'après les deux comptes dans la même room.

Les messages sont envoyés et reçus dans les deux compte en même temps.

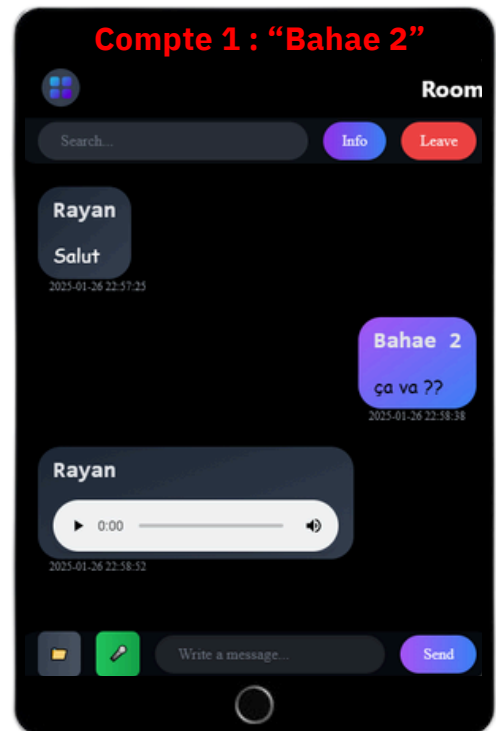


Figure 30 : Test des messages «compte1»

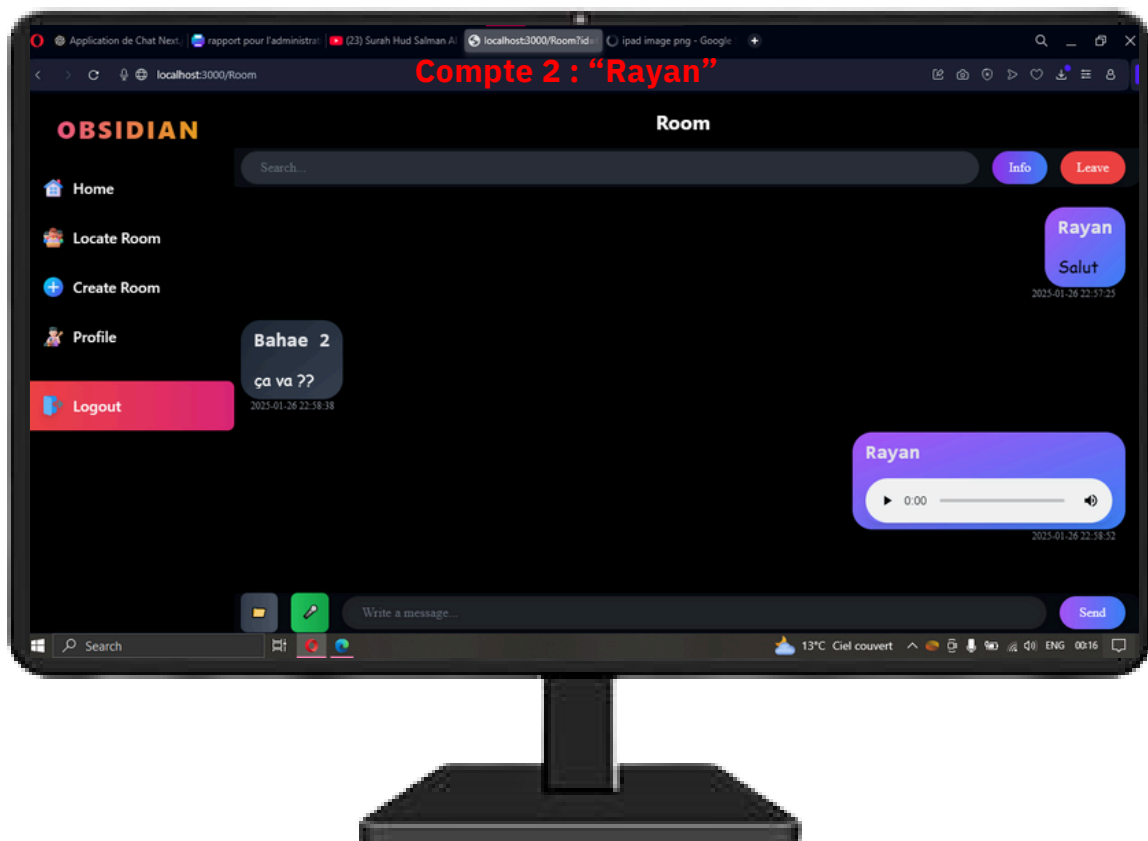


Figure 31 : Test des messages «compte2»

Support des audios, images et vidéos

Dans l'application, les messages peuvent être de différents types : texte, audio, image ou vidéo. Lorsqu'un utilisateur envoie un fichier multimédia, ce fichier est converti en **Base64** pour être facilement intégré dans la base de données MongoDB. Cette conversion transforme le fichier en une chaîne de caractères qui peut être directement stockée dans le champ correspondant du message.

Le stockage en Base64 présente l'avantage de simplifier l'intégration des fichiers dans la base de données, car il permet de garder l'ensemble des données sous une forme uniforme et textuelle.

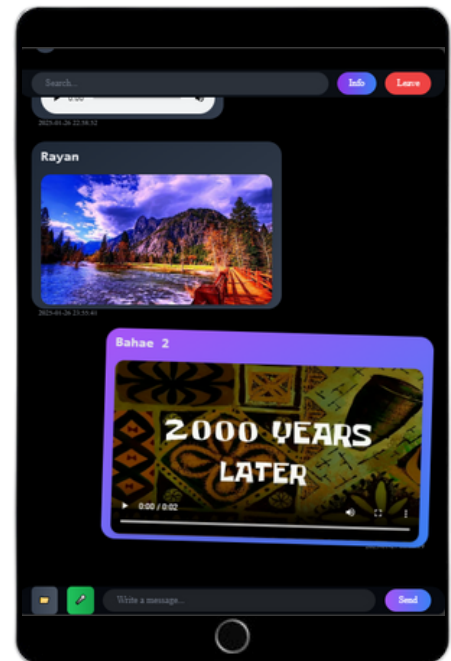


Figure 32 : Test des messages «messages multimédias»

Consulter les informations de la room :

Le bouton "Infos" fournit à l'utilisateur un accès rapide aux détails essentiels concernant la room. Lorsqu'il est cliqué, il ouvre une fenêtre modale ou une section dédiée affichant des informations clés telles que le nom de la room, la description, le nombre de membres et le rôle de l'utilisateur dans cette room (administrateur, membre standard, etc.). Ce bouton est particulièrement utile pour les utilisateurs qui souhaitent obtenir un aperçu rapide de la room sans interagir directement avec les messages ou les autres membres. Il permet d'améliorer l'accessibilité de l'information et offre une navigation fluide tout en maintenant l'interface épurée et claire.

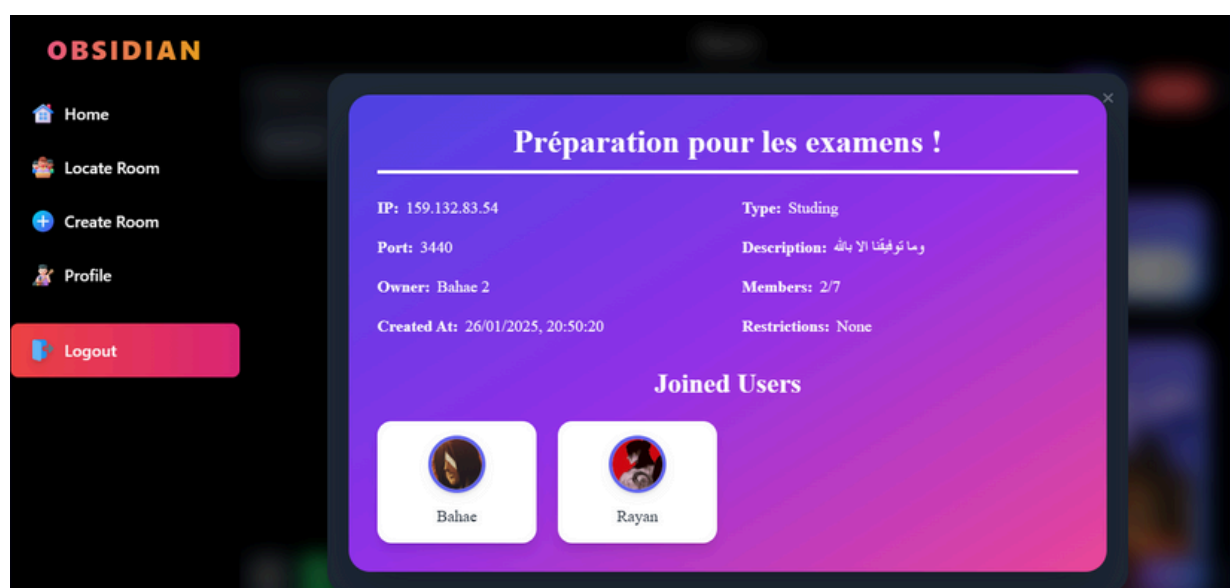


Figure 33 : Informations d'une room

Consultation de voir les profils des utilisateurs :

Dans la fenêtre des informations de la room, on peut cliquer sur un utilisateur pour voir son profile personel.

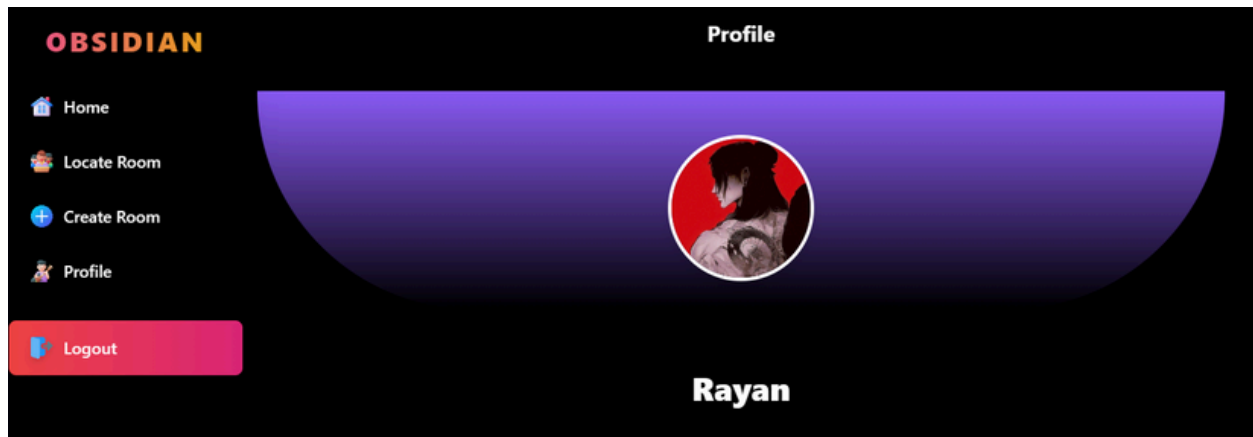


Figure 34 : Profile d'un utilisateur

Fonctionnalité de quitter la room :

Le bouton "Leave" permet à un utilisateur de se retirer d'une room dans laquelle il est actuellement membre. Lorsque l'utilisateur clique sur ce bouton un appel est effectué à l'API pour mettre à jour la base de données et retirer l'utilisateur de la room en question, décrementer son nombre de membres et supprimer l'accès de l'utilisateur aux messages et autres contenus liés à cette room. Après avoir quitté la room, l'utilisateur est automatiquement redirigé vers sa page d'accueil.

IV. Conclusion :

Le chapitre de réalisation de l'application a présenté les interfaces dédiées aux utilisateurs, démontrant ainsi l'aboutissement des efforts de conception pour offrir une expérience utilisateur satisfaisante et une gestion efficace des fonctionnalités de l'application.

Conclusion générale et perspectives

En conclusion, le projet OBSIDIAN a permis de développer une application de chat en temps réel, combinant des technologies modernes pour offrir une plateforme sécurisée, réactive et évolutive. L'objectif était de créer un espace numérique interactif, où les utilisateurs peuvent communiquer instantanément, gérer leurs sessions, et accéder à des fonctionnalités de messagerie enrichies (texte, images, vidéos, audio). Le choix des technologies, telles que React, Next.js, MongoDB, et Socket.io, a été guidé par la nécessité d'assurer une expérience fluide et une architecture robuste, capable de s'adapter à une croissance future.

L'application prend également en charge des fonctionnalités avancées, comme la gestion des rooms, la possibilité de créer, rejoindre ou quitter une room, ainsi que la gestion sécurisée des utilisateurs grâce à l'intégration de services comme NextAuth et Google One-Tap. Ce système garantit une sécurité optimale, en particulier lors de l'authentification et de la gestion des données personnelles. Les messages multimédia sont stockés sous forme de chaînes Base64 dans la base de données MongoDB, facilitant ainsi la gestion des différents types de contenus partagés au sein des rooms.

Enfin, ce projet s'inscrit dans une logique d'innovation et de simplicité, en visant à transformer la manière dont les utilisateurs interagissent dans un environnement numérique. En intégrant des fonctionnalités essentielles tout en offrant une interface intuitive et sécurisée, OBSIDIAN représente un outil moderne et performant, aligné sur les exigences actuelles des utilisateurs en matière de communication en ligne. Ce projet marque une étape importante dans le domaine de la messagerie web et ouvre la voie à de futures évolutions pour répondre aux besoins croissants des utilisateurs et à l'évolution technologique.

Bibliographie

Documentation officielle :

1. Next.js Documentation

- Titre : Next.js Documentation
- Source : Vercel
- Lien : <https://nextjs.org/docs>
- Une ressource essentielle pour apprendre et maîtriser le développement avec Next.js, incluant les concepts de rendu côté serveur, génération statique et gestion des routes.

2. NextAuth.js Documentation

- Titre : NextAuth.js Documentation
- Source : NextAuth.js
- Lien : <https://next-auth.js.org/getting-started/introduction>
- Guide officiel pour intégrer une authentification sécurisée avec NextAuth.js, en utilisant des fournisseurs comme Google, Facebook, ou encore des comptes personnalisés.

Vidéos YouTube (pour les autres technologies) :

1. MongoDB et Mongoose

- Titre : MongoDB Tutorial for Beginners | MongoDB Crash Course
- Chaîne : Programming with Mosh
- Lien : <https://www.youtube.com/watch?v=ofme2o29ngU>
- Cette vidéo explique les bases de MongoDB ainsi que l'utilisation de Mongoose dans les projets Node.js.

2. Socket.io

- Titre : Build a Real-time Chat Application with Socket.io
- Chaîne : Web Dev Simplified
- Lien : <https://www.youtube.com/watch?v=rxzOqP9YwmM>
- Une explication claire pour utiliser Socket.io afin de gérer la communication en temps réel.

3. NextAuth.js

- Titre : Next.js Authentication Tutorial with NextAuth.js
- Chaîne : The Net Ninja
- Lien : <https://www.youtube.com/watch?v=hkygjX2TMkM>
- Un tutoriel détaillé pour intégrer NextAuth.js avec Next.js pour une authentification sécurisée.

4. Base64 Encoding

- Titre : What is Base64 Encoding?
- Chaîne : Web Dev Simplified
- Lien : <https://www.youtube.com/watch?v=4ADu8nOfzCA>
- Une explication simple et claire sur l'encodage Base64 et son utilisation.